

Trabajo Final de Máster:
Aplicación de algoritmos de Machine
Learning para predicción de
rendimiento de acciones

HUGO GÓMEZ SABUCEDO

hugogomezsabucedo@gmail.com

Máster Big Data, Data Science & Inteligencia Artificial

Curso 2024-2025

Universidad Complutense de Madrid

Índice

1. Introducción al problema	4
1.1. Introducción y objetivos	4
2. Análisis descriptivo y transformaciones	5
3. Modelos: creación y aplicación	9
3.1. Elección de modelos	9
3.1.1. Modelos de árboles	10
XGBoost	10
LightGBM	11
3.1.2. Modelos secuenciales	11
LSTM	11
GRU	12
3.1.3. Modelos convolucionales	12
Conv1D	12
3.2. Preparación y configuración de modelos	12
3.2.1. Creación de datos supervisados	12
3.2.2. <i>Tuning</i> de hiperparámetros y métricas	13
3.2.3. Validación cruzada	14
3.2.4. Entrenamiento de los modelos	15
3.3. Resultados preliminares de los modelos	15
3.4. Bagging: aplicación y resultados	17
4. Resultados	20
5. Conclusiones	23
6. Bibliografía	23
A. Código EDA	24

B. Código para H=4	28
C. Código para H=1	41
D. Código combinado con H=1 y H=4	54

1. Introducción al problema

1.1. Introducción y objetivos

El análisis de datos e información financiera es una tarea altamente compleja ya que, al contrario de lo que ocurre en otras series temporales (por ejemplo, de datos de turismo o de viajes), no tienden a ser tan regulares, oscilando mucho ya no sólo de un día a otro, sino en una misma jornada. Además, a la hora de invertir, se busca obtener no sólo el máximo rendimiento en términos absolutos, sino de forma proporcional: es decir, ganar lo máximo posible, invirtiendo no mucho dinero. Es importante también tener en cuenta el posible riesgo de la inversión, así como el entorno que rodea a la misma: si invertimos en una empresa que viene creciendo a un ritmo vertiginoso, es posible que rápidamente deje de crecer o se estanque, con lo que el rendimiento obtenido no sería tan alto como el esperado. Por otra parte, invertir en una empresa que viene reduciendo su cotización no siempre puede ser una buena idea: si esta reducción se produce en un contexto global, podría ser una buena inversión, ya que lo más probable es que vuelva a crecer y se gane dinero; pero si esto se produce en un contexto local, lo más probable es que esa empresa no se recupere, o no lo haga tan rápidamente.

Es por ello que, para poder realizar estas predicciones, se propone este Trabajo de Final de Master, en el que se analizará un dataset disponible en Kaggle que contiene información financiera de las 500 empresas del índice bursátil SP&500 de Estados Unidos. Por tanto, el objetivo de este trabajo será, empleando estos datos, crear y comparar distintos modelos y técnicas de Machine Learning, encontrando el mejor de ellos para predecir el rendimiento de las acciones y poder decidir en qué empresa se debería invertir para obtener un mayor rendimiento.

Para ello, en primer lugar, se realizará una análisis descriptivo de los datos, y se transformarán y limpiarán para asegurar que tienen el formato correcto. Una vez tenemos el dataset adecuado, se crearán los distintos modelos (en este caso, se ha optado por combinar un XGBoost, LightGBM, LSTM, GRU y Conv1D) para entrenarlos sobre una submuestra de los datos, y se compararán las métricas de los mismos, para definir cuáles son los mejores. Sobre estos tres mejores, se aplicará un bagging, con el objetivo de reducir la varianza de los modelos y obtener uno más robusto. Una vez obtenido el modelo final, se reentrenará con los datos de todas las empresas, para poder finalmente realizar las predicciones.

2. Análisis descriptivo y transformaciones

El primer paso es, naturalmente, cargar los datos y realizar un análisis preliminar de los mismos. El código completo de este análisis se encuentra en el anexo A. En este análisis preliminar, en la figura 1 podemos ver que la serie contiene datos para un total de 491 empresas desde el 29 de noviembre de 2018 hasta el 29 de noviembre de 2023; es decir, un total de 5 años exactos. Si analizamos las empresas en función del número de datos que tienen vemos que en el top 10 de empresas con más registros todas tienen el mismo número, 1258, una cifra cercana a lo esperado (teniendo en cuenta que, si en estos cinco años de datos excluimos los fines de semana, días en las que los mercados financieros no abren, tendríamos un total de 1300 días, a los que habría que restar posibles festivos, etc.). Sin embargo, en el top 10 de empresas con menos datos, hay numerosas empresas con la mitad o más de sus datos faltantes. Posteriormente se explicará como se tratarán estos casos.

```
Empresas totales: 491
Rango de fechas: 2018-11-29 a 2023-11-29

Top 10 empresas con más registros:
Company
A      1258
NTR    1258
ORAN   1258
ON     1258
OKE    1258
ODFL   1258
O      1258
NXP    1258
NNG    1258
NYS    1258
Name: Date, dtype: int64

Bottom 10 empresas con menos registros:
Company
CPNG    686
COIN    663
GFS     525
NU      496
CEG     469
HLN     341
MBLY    275
GEHC    240
KVUE    145
ARM      54
Name: Date, dtype: int64
```

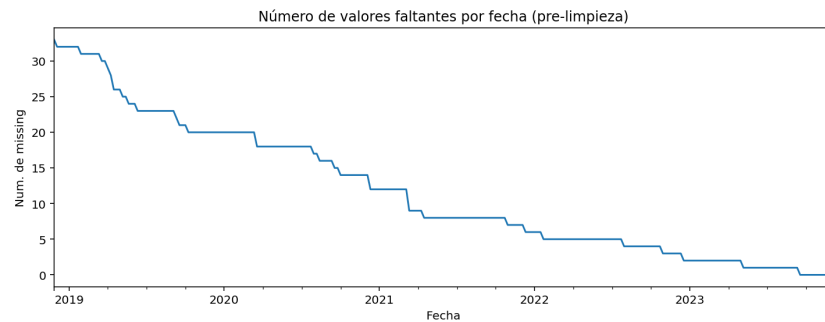
Figura 1: EDA preliminar

Visto que el dataframe carece de datos para los fines de semana, lo cual dejaría puntos en la serie temporal sin datos, se ha decidido hacer un resampleo de los mismos con una frecuencia semanal, calculando la media de los datos de esa semana. De esta forma, nos aseguramos de tener una serie de datos consistentes con la que trabajar. Este resampleo, sin embargo, presenta una desventaja, ya que perderemos cierta granularidad en los datos y la posibilidad de registrar mejor ciertas subidas o bajadas bruscas que se produzcan de un día para otro. Pero, por otra parte, como se ha comentado, se gana en consistencia en los datos y también en eficiencia computacional, ya que tenemos que tener en cuenta que el conjunto inicial de datos con el que trabajamos tiene alrededor de 603,000 filas. Además, se ha pivotado el dataframe, de forma que trabajemos con un conjunto de datos cuyo índice sea la fecha (en este caso, semana); las columnas, las distintas empresas; y vayamos almacenando los valores.

Con estos pasos, se ha reducido el dataset a uno de 262 filas por 491 columnas. El siguiente paso será analizar el porcentaje de valores missing por empresa y por fecha, como se ve en las figuras 2a y 2b. En este caso, observamos que el top 10 de empresas con más valores faltantes tiene un porcentaje de más del 40 %, y vemos que, según nos acercamos al presente, el número de missing por fecha va reduciéndose. Esto nos indica que son empresas que se

Company	
ARM	0.954198
KVUE	0.881679
GEHC	0.805344
MBLY	0.778626
HLN	0.729008
CEG	0.625954
NU	0.603053
GFS	0.580153
COIN	0.473282
RBLX	0.454198
dtype:	float64

(a) Valores missing por empresa



(b) Valores missing por fecha

Figura 2: Valores missing

ARM	KVUE	GEHC	MBLY	HLN	CEG	NU	GFS	COIN	RBLX	CPNG	SYM	ABNB	DASH
0.954198	0.881679	0.805344	0.778626	0.729008	0.625954	0.603053	0.580153	0.473282	0.454198	0.454198	0.454198	0.40458	0.40458

Figura 3: Empresas con NaN $\geq 40\%$

crearon posteriormente a la fecha de inicio de los datos, o que empezaron a cotizar en bolsa después de esta fecha. Por tanto, para tratar los valores missing, se usarán dos estrategias.

- En primer lugar, se ha definido un umbral máximo sobre el porcentaje de valores missing que puede tener una empresa, que en este caso se ha establecido como un 40 %, y se eliminarán por completo estas empresas. Esto se debe a que se considera que será complejo interpolar los datos para rellenar con los valores faltantes y que, además, no tendría mucho sentido. Como se ha visto, son empresas que han empezado a cotizar en bolsa en años intermedios del dataset, por lo que, si para más de la mitad de los años que analizamos no tenemos sus datos, no tiene mucho sentido incluirlas al análisis. Las empresas que se eliminarán se pueden ver en la figura 3.
- La otra estrategia, para las empresas que presenten menos de un 40 % de valores faltantes, se realizará una interpolación de los mismos. En este caso, indicando el índice como un eje temporal, se realiza una interpolación con *bfill* o *backward fill*. Es decir, se rellenan los valores missing con el siguiente valor válido hacia atrás, de forma que se usa el primer valor válido que haya después de los NaN para rellenarlos con ese valor.

Se han elegido estas estrategias, especialmente la de la interpolación temporal con *bfill*, ya que se han considerado las más adecuadas. Evidentemente, una interpolación con un valor fijo no tendría sentido, ya que distorsiona la serie, al igual que rellenar con medidas estadísticas como la mediana o la moda. Una interpolación polinómica podría no ser tampoco la más adecuada, ya que en series con mucha variabilidad o datos atípicos no funciona adecuadamente. Es por ello que se ha elegido la interpolación temporal, ya que mantiene la coherencia temporal de los datos, algo crucial. De esta forma, tras la aplicación de estas técnicas, tenemos unos datos perfectamente limpios.

Company	count	mean	std	min	25%	50%	75%	max
A	262	112.046	30.0516	61.0184	80.0922	118.561	134.803	176.669
AAPL	262	118.915	47.2635	35.5754	70.5751	130.659	155.707	195.309
ABBV	262	104.94	33.2663	53.1197	74.1516	100.88	138.726	163.54
ABEV	262	2.94515	0.619154	1.76905	2.54337	2.77898	3.34223	4.51929
ABT	262	98.2452	17.1848	61.8576	81.7266	102.317	110.96	135.703
ACGL	262	45.4913	15.6241	22.98	34.155	41.62	47.52	86.6
ACN	262	249.765	62.9826	127.917	190.243	263.051	301.962	402.575
ADBE	262	417.346	116.001	208.8	319.16	407.605	500.067	688.37
ADI	262	139.015	32.5626	75.217	107.107	147.315	164.421	197.481
ADM	262	58.0849	19.3557	27.946	38.7064	57.0652	75.3234	95.1963

Figura 4: Análisis descriptivo de las 10 primeras empresas

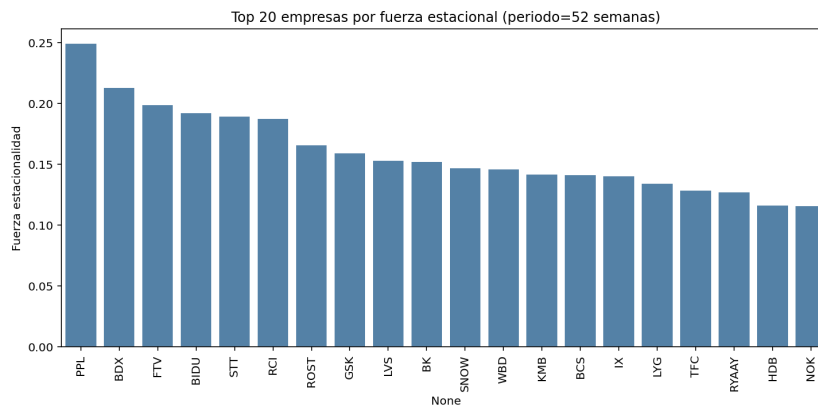


Figura 5: Empresas según su fuerza estacional

El siguiente paso es realizar un muy breve análisis descriptivo de los datos, en este caso para las 10 primeras empresas. Se puede observar una variabilidad en los precios, con empresas como AAPL (Apple) con desviaciones estándar elevadas teniendo en cuenta las medias. En general, las medias y medianas (50 %) son cercanas, lo que indica que, pese a las fluctuaciones, estas no son excesivamente elevadas. De todas formas, estos datos nos podrían indicar una ligera volatilidad, lo cual se debe tener en cuenta en los análisis futuros.

A continuación, se analiza la fuerza estacional de las series, para ver qué empresas presentan una mayor fuerza estacional. Para ello, se hace la descomposición estacional de la serie, teniendo en cuenta que trabajamos con series temporales anuales (un periodo de 52 semanas), y calculamos la fuerza estacional, que es la relación entre la varianza del componente estacional de la serie y la varianza total de la serie. Así, se obtiene el gráfico de la figura 5, donde se ven las 20 empresas con mayor fuerza estacional. Esta clasificación la emplearemos para calcular la descomposición de la serie en sus componentes, como se ve en la figura 10.

Se ha elegido la serie con mayor fuerza estacional (PPL, figura 6), con menor fuerza estacional (MCK, figura 8) y la mediana (SHEL, figura 7). Además, en la figura 10, podemos ver

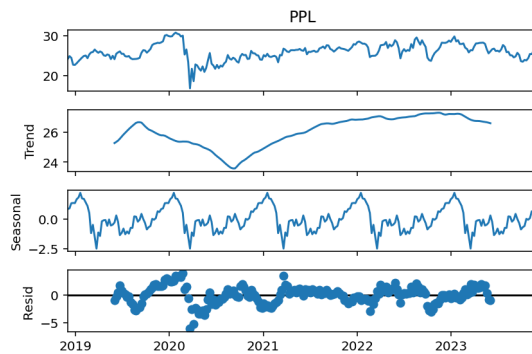


Figura 6: Descomposición estacional de PPL

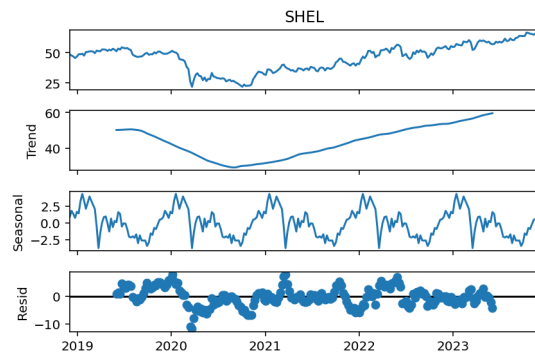


Figura 7: Descomposición estacional de SHEL

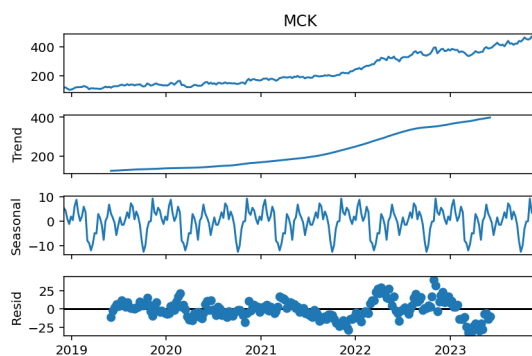


Figura 8: Descomposición estacional de MCK

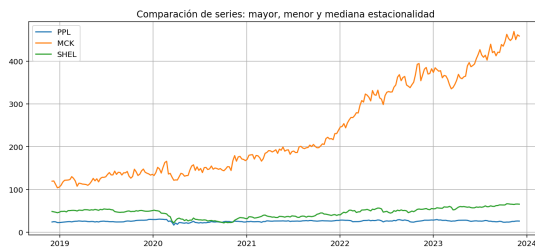


Figura 9: Comparación de las series

Figura 10: Descomposiciones estacionales

las tres series dibujadas juntas (aunque cada serie individual se puede ver también en su figura respectiva). La descomposición temporal de las series nos da cuatro gráficos distintos:

1. El **componente observado**, que es la serie temporal tal y como es, con todos sus valores.
2. El **componente de tendencia**, que captura la dirección a largo plazo de la serie. En este caso, se ve una tendencia creciente en las tres empresas, pero con ligeras diferencias. Mientras que la primera empresa se ve una tendencia creciente, que se vio interrumpida por la pandemia, para luego sufrir de nuevo un crecimiento explosivo y un estancamiento; en la segunda empresa se observa simplemente la caída debida a la pandemia y el posterior crecimiento, en una tendencia ascendiente que aún no se ha cortado; y en la última empresa, se ve un crecimiento constante, menor o menos abrupto los primeros años (coincidentes también con la pandemia).
3. El **componente estacional**, que captura los patrones o ciclos en los datos. Estos patrones pueden deberse a numerosas causas, como el clima, festividades, las estaciones, etc. Representa las fluctuaciones que ocurren, para el mismo período (en este caso, anual), en cada ciclo (por ejemplo, se observa que, por lo general, al inicio del año, los precios

sufren una bajada, precedida de una subida en los últimos meses del año). Es crucial para el análisis, ya que nos permite saber si podremos predecir estos patrones correctamente.

4. El **componente residual**, la variabilidad aleatoria que no se explica ni por la tendencia ni por la estacionalidad. Se corresponde con anomalías o eventos únicos, ruido, por lo cual, como es esperable, nuestras series presentan ciertos residuos. Sin embargo, lo importante es que estos estén incorrelados, que sean aleatorios. Si aplicamos la prueba de Ljung-Box sobre los residuos, que evalúa precisamente si existe una autocorrelación significativa, obtenemos p-valores extremadamente pequeños, del orden de 10^{-100} . En este caso, nos indicaría que los residuos no son aleatorios. Sin embargo, esto puede deberse a que esta descomposición se hace usando modelos relativamente simples, lineales, y que podríamos necesitar modelos más complejos para analizar series complejas como las que tenemos.

Por último, una vez que hemos realizado el análisis al completo y que hemos limpiado los datos, vamos a realizar una exportación del dataset ya limpio, para poder usarlo como base de trabajo en los sucesivos códigos y no tener que realizar la limpieza cada vez que se ejecutan.

3. Modelos: creación y aplicación

En esta sección se comentará el bloque principal de este trabajo: los modelos. Se explicará cuáles se han elegido y el porqué, comentando sus ventajas y desventajas. Se comentará también como se han creado y entrenado, así como los parámetros empleados, explicando las decisiones tomadas. Se analizarán los resultados y métricas obtenidas, y se explicará el proceso de bagging empleado para mejorar los modelos, con sus correspondientes métricas y resultados.

3.1. Elección de modelos

Para este trabajo se han elegido cinco modelos principales, cada uno de los cuales aporta numerosas ventajas y desventajas. Además, se han desarrollado tres scripts distintos como parte de la elección no sólo del mejor modelo, sino de la forma de predecir los datos. Por una parte, tenemos en el anexo B un código que entrena los modelos y realiza predicciones para una predicción a corto plazo, de una semana; por otra parte, en el anexo C, un código que realiza el entrenamiento y predicción en un horizonte a medio plazo de 4 semanas; y por último, en el anexo D, un framework combinado.

El motivo de haber desarrollado estos tres códigos, aunque realmente son similares entre ellos, es un enfoque progresivo del problema. Aunque en todos los casos se emplean los mismos modelos, la diferencia radica en el horizonte temporal de salida: mientras que en el caso de $H=1$ los modelos se centran en predecir un único paso, los modelos con $H=4$ deben predecir a un mes vista, cambiando el enfoque del problema a una predicción multihorizonte. Esto, pese a mantener la estructura de los modelos, incrementa la dificultad del aprendizaje de los mismos y suele afectar también a la precisión de las predicciones, ya que tienen el reto no sólo de predecir con mayor o menor exactitud el siguiente valor de la serie, sino la tendencia y evolución de la serie. Finalmente, el framework combinado se concibió como una manera de tener una mejor

visión de los resultados de los modelos, ya que evidencia cuál de los dos casos genera mejores predicciones. De este modo, la combinación de las tres versiones enriquece también el análisis.

En lo que a los modelos respeta, se han elegido cinco tipos diferentes: XGBoost, LightGBM, LSTM, GRU y Conv1D.

3.1.1. Modelos de árboles

Los modelos de árboles se basan en la combinación, mediante *gradient boosting*, de árboles de decisión “débiles” secuencialmente, de forma que cada árbol nuevo corrige los errores del anterior. Aprenden minimizando una función de pérdida y cada iteración añade un árbol entrenado sobre los residuos. Son buenos para capturar no linealidades e interacciones entre variables, pero no comprenden el orden temporal, por lo que les tendremos que proporcionar ventanas como features, como se explicará en 3.2. Dentro de este tipo se encuentran los modelos de XGBoost y LightGBM.

En el contexto de este trabajo, son adecuados ya que, tras la creación de las ventanas temporales, los datos presentan una estructura tabular, con la que cada fila representa un bloque de W semanas y la variable objetivo se corresponde con la semana siguiente (o las cuatro siguientes). Esto es fácilmente manejable por estos algoritmos, capturando las relaciones no lineales entre ellos.

XGBoost

XGBoost, o eXtreme Gradient Boosting, es un algoritmo de aprendizaje supervisado que implementa una técnica llamada *gradient boosting*. En esta técnica se entrenan secuencialmente múltiples árboles de decisión, de forma que cada nuevo árbol corrija los errores cometidos por los anteriores optimizando una función de pérdida. Su principal ventaja es que usa la memoria muy eficientemente y tiene una estructura optimizada para que las predicciones y las construcciones de los árboles sean fácilmente escalables. Esta técnica, además permite controlar el sobreajuste si se controlan o modifican adecuadamente los hiperparámetros, pero presenta la desventaja de necesitar ventanas muy largas para ver bien el pasado y puede ser muy costoso con árboles muy profundos.

Los parámetros que se pueden configurar para estos modelos, y los que se usarán en este trabajo, son tres:

1. **n_estimators**: representa el número de árboles; a mayor número de árboles, mejores resultados, pero también hay más riesgo de *overfitting*.
2. **max_depth**: representa la profundidad máxima de cada árbol, y permite limitar la complejidad.
3. **learning_rate**: representa cuánto aporta cada nuevo árbol; cuanto más pequeño es el valor, más robusto es el modelo, pero más lento de entrenar.

LightGBM

LightGBM, o Light Gradient Boosting Machine, es otra forma de implementación de boosting de árboles, diseñado con el objetivo de ser eficiente tanto en el tiempo que requieren para ser entrenados como en el uso de la memoria. Usa histogramas y crecimiento *leaf-wise* que, al contrario de los modelos tradicionales (o *depth-wise*, que trabajan dividiendo horizontalmente los árboles en cada nivel, generando árboles más anchos), trabaja seleccionando la hoja más prometedora (con mayor *gain*) para dividir el árbol. Esto acelera mucho la creación del árbol, pero también implica mayor riesgo de sobreajuste.

Para este modelo, se configurarán también los mismos parámetros que en anterior.

3.1.2. Modelos secuenciales

Estos modelos procesan los datos en un orden temporal, manteniendo un estado oculto o memoria. Aprenden combinando, en cada paso, la entrada actual con el estado anterior, produciendo un nuevo estado, con compuertas controlando qué información se mantiene o se olvida. Son muy buenas para capturar dependencias en series temporales, tanto a corto como a largo plazo, así como estacionalidades de las mismas. Sin embargo, son más costosas de entrenar y sensibles a escalados de los datos y a *overfitting*. Son los modelos más complejos. Dentro de este tipo se encuentran los modelos de LSTM y GRU.

En el contexto de este trabajo, con series con información de numerosos meses y en las que cambios repentinos en los datos son habituales, son modelos muy apropiados, ya que permiten tener en cuenta no sólo los datos de las semanas más inmediatamente recientes, sino también la trayectoria de la serie y su inercia. Esto presenta una ventaja importante sobre los modelos de árboles, permitiendo ajustar los datos con la información que tienen del largo plazo y saber si venían en una tendencia creciente o decreciente.

LSTM

Las LSTM, o Long Short-Term Memory, son un tipo de red neuronal recurrente (RNN) que incorporan una memoria a largo plazo con el objetivo de recordar información durante secuencias largas. Están compuestas por células, cada una de las cuales tiene dos estados: un estado oculto, que es el que se transmite de paso en paso y se usa para generar la salida; y un estado de celda (o cell state), que es el que actúa como memoria a largo plazo. El flujo de la información se controla mediante tres *gates*: la *forget gate*, que decide qué parte de la memoria anterior se descarta; la *input gate*, que decide qué nueva información se guarda; y la *output gate*, que controla qué parte de la memoria influye en la salida.

Los parámetros que se ajustarán para este modelo son:

1. **units**: el número de células ocultas de la red.
2. **dropout**: una forma de regularización, que inactiva ciertas neuronas para que las restantes aprendan de forma más robusta.

GRU

Las GRU, o Gated Recurrent Unit, son una variante simplificada de las LSTM, que mantienen tanto la idea de la memoria como de las puertas, pero fusionando algunas de ellas, con lo que logran reducir los parámetros y acelerar el entrenamiento. Cada unidad GRU tiene ahora un único estado oculto, controlado por dos puertas: la *update gate*, que decide cuánto se mantiene del estado anterior; y la *reset gate*, que decide cuánto se olvida. En este caso, también se ajustarán los mismos parámetros que para las LSTM.

3.1.3. Modelos convolucionales

Estos modelos presentan una complejidad intermedia entre los modelos de árboles y los secuenciales. Su idea básica es aplicar filtros o kernels que van barriendo y analizando la secuencia y detectando patrones locales, bien sean tendencias u oscilaciones, en las ventanas. Aprenden usando pequeñas cuadrículas, llamadas *kernels* o núcleos, que se van desplazando sobre los datos y generando mapas de activación. Su ventaja es que permiten combinar varias capas, cada una detectando un patrón o tendencia específica. Tienen la ventaja de capturar muy bien patrones locales repetitivos y de corto plazo, además de ser rápidos y fácilmente paralelizables. Dentro de este tipo de modelos se usará el Conv1D.

Conv1D

Las Conv1D, o Convolutional Neural Network 1D, son un tipo de red neuronal que, en lugar de procesar los datos paso a paso, aplican filtros deslizantes que los van recorriendo y detectando patrones locales. Funcionan con filtros convolucionales, que no son más que vectores de pesos que se desplazan sobre la secuencia de estado y van calculando, para cada posición, una combinación lineal de los valores de la ventana. Cada filtro aprende un patrón específico (que puede ser tendencias ascendentes en el último mes, por ejemplo) de forma que, combinando el uso de varios filtros en paralelo, se aprenden numerosos patrones. Tras la convolución, se usan capas adicionales para aplanar y combinar las características.

Para este modelo se ajustarán, al igual que los anteriores, las **units** y el **dropout**, así como el **kernel_size**, que modifica la longitud del kernel en pasos de tiempo.

3.2. Preparación y configuración de modelos

3.2.1. Creación de datos supervisados

Un aspecto crucial a tener en cuenta a la hora de comenzar a trabajar con los modelos es la necesidad de transformar los datos en un problema supervisado, lo que se conoce como *ventanización* o *windowing*. Esto es necesario porque los modelos que se entrenan requieren recibir los datos en forma de pares de entrada-salida, y la serie de datos en crudo (una simple columna de precios a lo largo del tiempo) no lo es. De esta forma, el dataset restante tendrá la siguiente forma: $X_t = [y_{t-W+1}, y_{t-W+2}, \dots, y_t]$

Para los modelos de árboles, ya que no entienden de orden temporal, si no se proporcionan

los datos en forma de ventana sólo verían una columna de valores sin contexto. En el caso de las RNN, aunque pueden procesar secuencias, necesitan un tamaño fijo de entrada, con el que saber cuántos pasos para atrás mirar para hacer la predicción. Para esto, se ha realizado una función que, en base al tamaño de ventana definido y al horizonte de predicción, genera unos nuevos datos donde la entrada son las últimas W observaciones, y la salida es 1 o 4 (en función de H) observaciones futuras.

Respecto a la elección del tamaño de la ventana, se ha optado por un tamaño de 25 semanas (aproximadamente 6 meses), ya que se ha considerado un tamaño equilibrado entre la captura de suficiente contexto histórico para reflejar correctamente las tendencias a largo plazo y la reducción del ruido. Una ventana muy corta, de unas 3-5 semanas, haría que el modelo sólo observase fluctuaciones recientes, y que no detectase correctamente las tendencias. Una ventana muy larga, de por ejemplo 100 semanas, incluiría datos demasiado antiguos, que diluiría la información más reciente (crucial, para detectar tendencias inmediatas) y ralentizaría mucho el entrenamiento. Es, por tanto, un punto intermedio. Además, para el horizonte de predicción que se ha definido (bien sea el de una semana o el de cuatro), es un tamaño adecuado, ya que no se necesita información, por ejemplo, de dos años, para predecir el rendimiento de la próxima semana.

3.2.2. *Tuning* de hiperparámetros y métricas

Como se comentó en 3.1.1, cada uno de los modelos que se emplean tienen una serie de hiperparámetros que condicionan no sólo su capacidad de aprendizaje, sino cómo se comportan frente al sobreajuste de los datos o qué límite tienen a la hora de parar. A diferencia de los parámetros internos que tienen cada uno de ellos, que se van ajustando automáticamente, los hiperparámetros se deben establecer de forma externa.

Para ello, se ha aplicado una *grid search*, en la que se define un espacio discreto de valores para cada uno de los posibles hiperparámetros y se entrena el modelo sucesivamente con cada una de las combinaciones. Este es un enfoque que, si bien es computacionalmente costoso, permite explorar una serie de valores razonables, obtenidos a partir de experimentos realizados tanto sobre los mismos datos como analizando los valores que se suelen usar más frecuentemente. Cabe destacar que los valores que se presentan en los códigos de los anexos no son todos con los que se ha probado, ya que previamente se ha hecho una búsqueda más extensa, con el objetivo de ajustarlos y eliminar aquellos con peores resultados.

Relacionado con los hiperparámetros, conviene hablar de las métricas empleadas para medir el rendimiento de los modelos. En este caso, se ha definido una función propia que computa a la vez distintas métricas para el mismo modelo:

- R^2 (Coeficiente de determinación). Es la métrica por excelencia, que mide la proporción de la varianza de la variable dependiente que es explicada por el modelo. Valores próximos a 1 indican que el modelo explica perfectamente los datos, mientras que próximos a 0 indican que el modelo no mejora respecto a predecir siempre la media. Es el criterio que usaremos como principal a la hora de determinar el mejor modelo.
- MSE (Mean Squared Error). Se calcula como la media de los cuadrados de las diferencias

entre los valores reales y los predichos, y es útil ya que penaliza fuertemente los errores grandes (debido a la elevación al cuadrado), aunque no es interpretable en las mismas unidades que los datos originales.

- **MAE** (Mean Absolute Error). Similar al anterior, pero mide únicamente la media de las diferencias absolutas. Es más robusto frente a valores atípicos o excesivamente altos, e indica, en promedio, cuánto se desvía la predicción de los valores reales, con la ventaja de hacerlo en las unidades de la variable.
- **RMSE** (Root Mean Squared Error). Es simplemente la raíz cuadrada del MSE, lo que lo hace más interpretable al devolver el error medio, ahora sí, en las mismas unidades que los datos.
- **MAPE** (Mean Absolute Percent Error). Mide el error relativo en porcentaje. Tiene la ventaja de ser intuitivo (por ser un porcentaje), pero no funciona bien con valores próximos a cero. En este caso, nos sirve para comparar empresas con diferentes escalas de precios.
- **EVS** (Explained Variance Score). Mide cuánta varianza de los datos es explicada por el modelo, de forma similar al R^2 , pero más centrada en la dispersión de los errores.
- **DA** (Directional Accuracy). Se emplea en algunos casos para evaluar si el modelo predice correctamente la dirección del cambio (es decir, si el modelo acierta cuando la serie sube o baja respecto al valor anterior). Es muy útil, ya que en nuestro caso seguramente sea más útil predecir bien la tendencia que el valor exacto.

3.2.3. Validación cruzada

A la hora de evaluar los modelos, es importante comprobar que las métricas que hemos obtenido reflejen su capacidad de generalización a datos que no se le han proporcionado como conocimiento. En problemas de regresión o clasificación, suele ser suficiente con realizar una validación cruzada de los datos, dividiéndolos en particiones de entrenamiento y de test. Sin embargo, en datos con una estructura temporal, como los nuestros, esta validación cruzada es inapropiada, ya que rompería la propia secuencialidad de los datos, produciendo un *data leakage* (cuando los modelos se entrenan con información futura).

Es por ello que, en este caso, se ha optado por emplear una validación cruzada, pero temporal (denominada `TimeSeriesSplit`). Este método genera divisiones sucesivas de los datos, en donde cada conjunto de entrenamiento contiene únicamente observaciones pasadas, y el conjunto de validación contiene valores futuros, que no se emplean en el entrenamiento. Así, combinamos el respeto por el orden cronológico y se simulan condiciones reales de predicción.

En este caso, se han empleado un número variable de particiones en función del tipo de modelo con el que se trabaja. Para modelos de árboles de decisión, debido a que presentan un menor coste computacional, se han utilizado cinco particiones; mientras que para los modelos secuenciales y convolucionales, de mayor complejidad, se han empleado únicamente tres particiones. Cada partición se evalúa con las métricas comentadas en 3.2.2, y los resultados obtenidos se corresponden con la media de las métricas a lo largo de todos los *folds*.

3.2.4. Entrenamiento de los modelos

Este proceso es la parte central del trabajo, pues es donde se entrenan los algoritmos para que identifiquen los patrones y realicen las predicciones. Como se comentó en 3.2.1, se emplearán estos datos transformados, tanto para los casos en el que se emplea el horizonte de 1 semana como para el de 4, ya que se ha diseñado una función que crea los datos teniendo en cuenta ambos parámetros. Sin embargo, estos datos no son los que se empleen directamente para entrenar los modelos, sino que primero deben escalarse, empleando un `StandardScaler`. Esto se hace para homogeneizar las escalas de los datos, y evitar que empresas con precios de las acciones muy elevados dominen los modelos y tiren de las predicciones. Además, se realiza un aplanado de los datos en vectores para los modelos de árboles, mientras que se conservan como tensores tridimensionales para las redes neuronales, para ser consistentes con las dimensiones de los datos que necesitan cada uno de los modelos.

Una particularidad a destacar de esta fase de entrenamiento es que el entrenamiento y ajuste inicial de los hiperparámetros se realiza inicialmente sobre un subconjunto aleatorio de un 40 % de las compañías. Esto responde a varios motivos. Por una parte, eficiencia computacional; dado que estamos trabajando con aproximadamente 500 empresas, entrenar cada uno de los modelos (en este caso, y con la criba de hiperparámetros que ya hemos hecho, entrenamos 8 modelos tanto en XGBoost y LightGBM, 6 en LSTM y GRU y 12 en Conv1D) con todos los datos consumiría muchos recursos, tanto a nivel computacional como temporal. Por otra parte, al seleccionar de forma aleatoria las empresas nos aseguramos de que sea una muestra heterogénea (en vez de elegir, por ejemplo, el top de empresas con mayor cotización u otro criterio).

Una vez ajustados todos los modelos sobre este subconjunto, podemos elegir los más prometedores en términos de sus hiperparámetros, para aplicarles el bagging y poder reentrenarlos de nuevo, ahora sí, con todos los datos. Además, no corremos el riesgo de que se produzca un sobreajuste en los datos en el modelo final, ya que esta fase inicial únicamente se usa para elegir el modelo más prometedor, y luego reentrenamos el modelo, ya sí, teniendo en cuenta todos los datos.

3.3. Resultados preliminares de los modelos

En este apartado, analizaremos los resultados preliminares obtenidos entre los cinco modelos, para determinar los mejores, y aquellos a los que se les aplicará el *bagging*. Para ello, compararemos las métricas que se explicaron en 3.2.2, veremos si hay relaciones entre los modelos y aplicaremos *bagging* a los mejores.

Así, en el cuadro 1 podemos ver los resultados de las métricas para las predicciones realizadas con un horizonte de una semana, mientras que en el cuadro 2 podemos ver las métricas para las predicciones realizadas con un horizonte de cada semana, ordenados en ambos casos por el R^2 . Se observa un patrón claro, donde se ve que Conv1D es el mejor en ambos casos, con mucha diferencia sobre el resto; los modelos de árboles (LightGBM y XGBoost) se sitúan un peldaño por debajo, también con buenas métricas; mientras que los peores son los modelos secuenciales, tanto LSTM como GRU, que, en diferente orden, presentan el peor

Modelo	MSE	RMSE	MAE	R2	MAPE	EVS
Conv1D	1093.488254	30.505075	11.599173	0.981727	10.36545	0.981755
XGBoost	94997.178863	147.651011	31.917192	0.854591	5.743488	0.859903
LightGBM	95040.103876	148.86174	31.931191	0.852904	4.781479	0.858218
GRU	148093.259157	274.145885	61.88736	0.43245	16.99539	0.468539
LSTM	151292.014625	276.373591	63.479611	0.425963	18.0122	0.455708

Cuadro 1: Resultados de los modelos (H=1)

Modelo	MSE	RMSE	MAE	R2	MAPE	EVS
Conv1D	1482.329199	35.31938	13.317512	0.975987	12.895549	0.976049
XGBoost	95709.451337	151.978414	34.332828	0.847299	7.828946	0.85269
LightGBM	96230.201717	153.48642	34.53518	0.844269	7.768551	0.849684
GRU	148269.588843	273.933054	62.878443	0.429544	14.947863	0.465223
LSTM	148871.35456	275.026711	64.14006	0.422786	19.796763	0.461598

Cuadro 2: Resultados de los modelos (H=4)

rendimiento.

El modelo Conv1D se muestra como el más preciso en ambos horizontes, tanto en H=1 (con un R^2 de 0.981) como en H=4 (con un R^2 de 0.975), lo cual significa que explica más del 98 % de la varianza de los datos, lo cual refleja un buen ajuste. Además, tiene un error absoluto y relativo bastante bajo, de un 11.5 % y un 10.4 % respectivamente, los más bajos de entre todos los modelos, lo que significa que las predicciones son muy cercanas a los resultados reales. Esto se refuerza con el alto valor de la EVS, del 98 %.

Este era el resultado esperable, especialmente en el contexto de este trabajo, ya que las convoluciones son muy útiles para capturar los patrones locales de los datos de forma muy eficiente en series temporales, sin necesidad de tener dependencias de largo plazo muy largas. Además, con una arquitectura más sencilla y menos propensa al sobreajuste, se facilita la generalización, lo que explica que el rendimiento sea prácticamente idéntico en ambos horizontes.

Sin embargo, hablando de sobreajuste, unas métricas tan perfectas nos podría dar lugar a pensar que el modelo haya caído en un sobreajuste de los datos, sobre todo teniendo en cuenta la comparativa con el resto de modelos. Este tema se analizará más en 3.4, pero hay varios indicios que nos indican que podría no haber sobreajuste. Se ha realizado una validación cruzada temporal, y el rendimiento se mantiene alto tanto en entrenamiento como en test. Además, el comportamiento del modelo es consistente en ambos horizontes temporales, lo que indica una estabilidad del rendimiento, que sería más difícil en un modelo con un horizonte temporal más amplio. Como se ha comentado, los modelos convolucionales son especialmente buenos para capturar patrones cortos, con tendencias locales y cambios abruptos, que precisamente son la naturaleza de las series financieras.

En el otro extremo de rendimiento tenemos a los modelos secuenciales, tanto LSTM como GRU, que ofrecen un rendimiento abrumadoramente inferior en ambos escenarios, con un valor del R^2 que apenas llega al 0.4. Las métricas de error confirman esto también, con un MAE en torno a 60 y un MAPE en torno al 20 %. Estos modelos están pensados para manejar

dependencias a largo plazo, por lo que su mal desempeño podría ser previsible, ya que los patrones que encontramos en las series financieras suelen ser más a corto plazo, lo que no beneficia a estos modelos. Además, son muy sensibles al *tuning* de los hiperparámetros, que debe ser muy fino, ya que si no disponen de una configuración óptima tenderán o bien a sobreajustar o, como es este caso, a no converger adecuadamente. Es posible que, con una búsqueda más exhaustiva de hiperparámetros, analizando muchas más combinaciones, se llegara a obtener modelos un poco mejores, pero esto no ha sido posible, en parte, debido a la falta de recursos computacionales.

Por último, con un buen rendimiento, aunque un escalón por debajo de los modelos convolucionales, tenemos a los modelos de árboles, LightGBM y XGBoost, que obtienen unos resultados más que aceptables. Con un R^2 que se sitúa en torno al 0.85, además en ambos horizontes de predicción, tienen un muy buen rendimiento. Un hecho destacable es que, pese a tener unas métricas de error mucho más elevadas que el modelo convolucional (un RMSE, en $H=1$, de aproximadamente 150 frente al 30 que tenía Conv1D, y un MAE de 31 frente a 11), el MAPE o error porcentual es inferior, aproximadamente la mitad (para Conv1D teníamos un 10.36 % en $H=1$, mientras que el de XGBoost es de 5.74 %, y el de LightGBM es aún más pequeño, de 4.78 %). Esto nos indica que, aunque los errores son mayores en términos absolutos, en término relativo se equivoca menos. Teniendo en cuenta que los datos son el valor en dólares de las cotizaciones de las empresas, quiere decir que se equivocan más en empresas con valores más altos, donde el impacto relativo es menor (es decir, no es lo mismo equivocarse por 10 dólares en una acción que cotiza a 1000, que sería un 1 % de error; que hacerlo en 1 dólar en una acción que cotiza a 5, que sería un 20 %).

Este comportamiento también era esperable, ya que estos algoritmos captan muy bien relaciones no lineales entre los datos y patrones complejos, que son los que se producen en este caso. Sin embargo, frente a horizontes más largos, pueden perder precisión, como se ha observado, ya que al no modelar de forma nativa la secuencia temporal hace que pierdan parte de la información estructural de la serie.

Una vez analizados todos los resultados, se elegirán para aplicar el bagging el modelo de Conv1D, así como el de XGB, por ser los que mejores métricas presentan.

3.4. Bagging: aplicación y resultados

Con los dos mejores modelos seleccionados, se aplicarán técnicas de ensamblado de modelos con el objetivo de combinar los modelos de forma que se mejoren las predicciones realizadas por los mismos. Aunque en este caso tenemos un modelo, el Conv1D, que ya tiene unas métricas buenas, nos servirá también para ver si el modelo está sobreajustando o no. El *bagging*, también conocido como *bootstrap aggregation*, es una técnica consistente en, una vez entrenados los modelos de forma independiente, promediar sus predicciones, para obtener un modelo combinado que reduzca la varianza de las predicciones, mejorando su capacidad de generalización. Al combinar resultados de diversos modelos, se atenúan los errores propios de cada uno de ellos, generando predicciones estables y con mejor generalización, ya que se disminuye el riesgo de sobreajuste. Además, mejora la robustez, ya que se evita depender excesivamente de los sesgos de un modelo concreto. Su principal ventaja es su sencillez de implementación, y la notable mejora que proporciona en los resultados de los modelos. A

cambio, conlleva un mayor coste computacional y, además, puede ser innecesario en modelos que ya obtienen buenos resultados, como el nuestro. Por lo tanto, se deberá analizar si mejora los resultados, cuánto, y si esta mejora compensa el coste.

En este estudio, se ha aplicado en dos niveles. Por una parte, el bagging intra-modelo, en el que se ha entrenado cada modelo de forma independiente un total de 5 veces (n_bags). Esto, en modelos como Conv1D, es especialmente relevante, debido a que su entrenamiento es estocástico, dependiendo mucho del azar procesos como la inicialización aleatoria de pesos, el orden de los *minibatches* o la aplicación del *dropout*; por lo que al promediar los resultados de las salidas obtenemos predicciones más estables. En el caso de los modelos de árboles, que son más deterministas, esto no tiene tanto efecto, ya que se fija una semilla para la aleatoriedad de los datos.

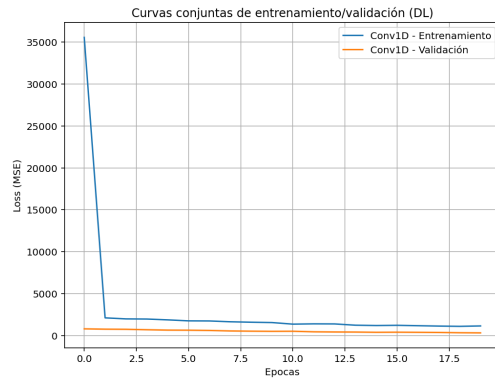
Por otra parte, el bagging inter-modelo, es decir, integrando los resultados de los diferentes modelos. Esto permite, como comentamos, complementar dos modelos de naturalezas diferentes, ya que por una parte capturamos relaciones no lineales de los datos y, por otra, patrones locales. Además, se han aplicado mecanismos específicos para controlar el sobreajuste:

1. **EarlyStopping**: este mecanismo controla la evolución de la pérdida en el conjunto de validación, de forma que se detiene el entrenamiento si esta deja de mejorar durante un número determinado de épocas consecutivas, evitando que la red siga ajustándose en exceso a los datos de entrenamiento.
2. **ReduceLROnPlateau**: permite reducir automáticamente la tasa de aprendizaje si se detecta una estabilización de la pérdida, lo que facilita el refinamiento del aprendizaje y evita estancamientos.

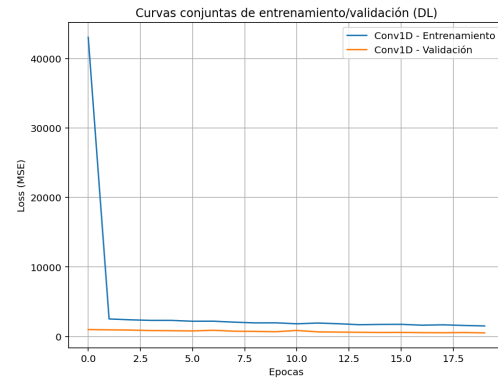
Para validar los resultados del *bagging*, además de emplear las mismas métricas que se emplearon en los casos anteriores, analizaremos también las curvas de pérdida de entrenamiento y de validación para Conv1D, ya que su ajuste progresivo de parámetros mediante retropropagación nos permite monitorizarlo. En este caso, tenemos la figura 11, que muestra en 11a las curvas para el horizonte de predicción semanal; y en 11b las curvas para el horizonte de predicción a cuatro semanas. La curva de entrenamiento representa el error del modelo al evaluarlo sobre los datos de train, y una disminución progresiva del mismo indica que el modelo aprende representaciones útiles de los datos. Por su parte, la curva de validación representa el error del modelo sobre datos no vistos (de test), y su evolución es crucial para analizar la capacidad de generalización. Estas curvas se analizan juntas, de forma que si ambas descienden de forma progresiva y se estabilizan en niveles bajos tendremos un modelo que generaliza bien. Si ambas permaneciesen altas, sin descender, tendríamos un caso de subajuste, donde el modelo no generaliza bien; si la de validación aumentara mientras se reduce la de entrenamiento, sería una señal inmediata de sobreajuste. Además, también podemos medir el sobreajuste viendo la distancia entre ambas curvas, siendo una distancia pequeña señal de que el modelo es bueno; mientras que una distancia grande indica *overfitting* (y, si se combina con valores muy elevados, indicaría *underfitting*).

En nuestro caso, vemos que la curva de entrenamiento desciende abruptamente en las primeras épocas para luego estabilizarse (pasa de 36000 a aproximadamente 2000, y se mantiene); mientras que la curva de validación, aunque se reduce, se mantiene estable desde el principio

en un rango que no supera 1000. Esto indica que el modelo ha aprendido patrones relevantes de los datos (por la reducción significativa del *train loss*), a la vez que presenta buena capacidad de generalización (por la estabilidad en el *validation loss*) y los valores bajos que presenta. Aun así, podrían aplicarse técnicas adicionales para reducir aún más el *train loss* y su brecha respecto al *validation loss*.



(a) Curvas train-val loss para H=1



(b) Curvas train-val loss para H=1

Figura 11: Curvas train-val loss

Por último, analizaremos las métricas del modelo final de bagging (tabla 3) y las compararemos con las que obtuvimos con los modelos base, tanto en la tabla 1 como en la tabla 2. En general, vemos que para el horizonte de predicción a corto plazo mejora los modelos, mientras que en horizontes más largos tiene un rendimiento más estable. En H=1, el R^2 aumentó ligeramente, desde el 0.9817 del Conv1D al 0.9882, pero con una destacable disminución de los errores, que se reducen a la mitad, pasando el RMSE de 30.50 a 16.26 y el MAE de 11.60 a 5.57. Para H=4, el R^2 se mantiene prácticamente estable, ya que únicamente baja de 0.9760 a 0.9757, y los errores apenas varían, pasando el RMSE de 35.32 a 34.87 y el MAE de 13.31 a 13.16. De esta forma, vemos que el bagging, en general, ofrece mayor robustez y estabilidad, aportando más ventajas en horizontes cortos, y manteniéndose más estable en horizontes largos.

Horizonte	MSE	RMSE	MAE	R2	MAPE	EVS
H1	264.543	16.2648	5.5755	0.9882	6.2485	0.9882
H4	1421.3568	34.8701	13.1624	0.9757	10.8791	0.9757

Cuadro 3: Resultados del modelo con bagging

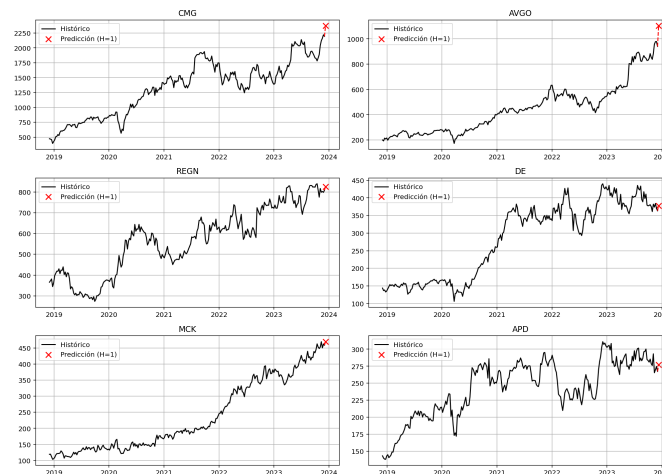


Figura 12: Predicciones para horizonte H=1

4. Resultados

Una vez aplicadas las distintas técnicas para obtener los mejores modelos, ya tenemos decidido aquel que usaremos para realizar las predicciones. Pero, si recordamos, como explicamos en 3.2.4, habíamos entrenado los modelos inicialmente sobre un subconjunto de los datos, con el objetivo de agilizar las pruebas, y poder entrenar únicamente el modelo final con todos los datos de todas las empresas. Una vez realizadas las predicciones, se obtienen los siguientes resultados.

```

1 Top 6 empresas con mayor crecimiento previsto (H=1):
2 CMG : Actual= 2194.45 | Pred= 2370.56 | DifAbs= 176.11 | Dif%= 8.03 %
3 AVGO: Actual= 940.53 | Pred= 1103.11 | DifAbs= 162.58 | Dif%= 17.29 %
4 REGN: Actual= 808.89 | Pred= 826.28 | DifAbs= 17.39 | Dif%= 2.15 %
5 DE : Actual= 363.31 | Pred= 378.08 | DifAbs= 14.77 | Dif%= 4.07 %
6 MCK : Actual= 458.51 | Pred= 468.86 | DifAbs= 10.35 | Dif%= 2.26 %
7 APD : Actual= 267.40 | Pred= 276.94 | DifAbs= 9.54 | Dif%= 3.57 %

```

En el caso de H=1, vemos que la empresa más rentable para invertir sería AVGO (Broadcom Inc.), ya que es la que proporciona una mayor crecimiento relativo, con un 17.29 % de incremento pronosticado. En lo que al crecimiento absoluto se refiere, la mejor empresa sería CMG (Chipotle Mexican Grill, Inc.), que nos daría un rendimiento de 176\$, pero tendríamos que invertir más dinero. En el gráfico 12 podemos ver estas seis empresas, graficadas su serie histórica y las predicciones en color rojo.

```

1 Top 6 empresas con mayor crecimiento previsto (H=4):
2 Empresa: ABEV. Ultimo valor: 2.74
3 +1 semana(s): Pred=4.83 | Dif=2.08 | Dif%=76.04 %
4 +2 semana(s): Pred=4.89 | Dif=2.14 | Dif%=78.22 %
5 +3 semana(s): Pred=5.03 | Dif=2.29 | Dif%=83.53 %
6 +4 semana(s): Pred=5.14 | Dif=2.40 | Dif%=87.47 %

```

7	
8	Empresa: MFG. Ultimo valor: 3.40
9	+1 semana(s): Pred=5.12 Dif=1.71 Dif %=50.34 %
10	+2 semana(s): Pred=5.11 Dif=1.70 Dif %=50.00 %
11	+3 semana(s): Pred=5.22 Dif=1.81 Dif %=53.21 %
12	+4 semana(s): Pred=5.33 Dif=1.92 Dif %=56.47 %
13	
14	Empresa: NOK. Ultimo valor: 3.58
15	+1 semana(s): Pred=5.16 Dif=1.58 Dif %=44.26 %
16	+2 semana(s): Pred=5.16 Dif=1.58 Dif %=44.01 %
17	+3 semana(s): Pred=5.27 Dif=1.69 Dif %=47.09 %
18	+4 semana(s): Pred=5.38 Dif=1.80 Dif %=50.28 %
19	
20	Empresa: TEF. Ultimo valor: 4.21
21	+1 semana(s): Pred=5.37 Dif=1.16 Dif %=27.67 %
22	+2 semana(s): Pred=5.38 Dif=1.17 Dif %=27.72 %
23	+3 semana(s): Pred=5.44 Dif=1.23 Dif %=29.31 %
24	+4 semana(s): Pred=5.56 Dif=1.35 Dif %=32.04 %
25	
26	Empresa: WIT. Ultimo valor: 4.78
27	+1 semana(s): Pred=5.92 Dif=1.14 Dif %=23.77 %
28	+2 semana(s): Pred=5.94 Dif=1.16 Dif %=24.34 %
29	+3 semana(s): Pred=6.00 Dif=1.22 Dif %=25.43 %
30	+4 semana(s): Pred=6.02 Dif=1.24 Dif %=25.92 %
31	
32	Empresa: NWG. Ultimo valor: 5.29
33	+1 semana(s): Pred=6.36 Dif=1.07 Dif %=20.21 %
34	+2 semana(s): Pred=6.37 Dif=1.08 Dif %=20.34 %
35	+3 semana(s): Pred=6.43 Dif=1.14 Dif %=21.55 %
36	+4 semana(s): Pred=6.40 Dif=1.11 Dif %=21.04 %

En el caso de $H=4$, observamos que la empresa más rentable para invertir sería ABEV (Ambev S.A.), ya que es la que muestra un mayor crecimiento relativo, con un 87.47 % de incremento pronosticado. Si atendemos al crecimiento absoluto, la mejor opción sería MFG (Mizuho Financial Group, Inc.), con un aumento proyectado de 1.92\$, aunque el importe de partida es menor que en otras compañías. En el gráfico 13 se representan estas seis empresas, mostrando en negro la serie histórica y en rojo las predicciones correspondientes a las cuatro semanas posteriores. Cabe destacar, además, que las predicciones para este muestran un crecimiento inicial más abrupto, estabilizándose en las semanas posteriores, lo que indica que no es el modelo más indicado.

Por último, analizamos el gráfico que resulta de realizar la predicción directamente con el horizonte $H=4$, frente a realizarla semana a semana, y retroalimentando el modelo con las nuevas predicciones. Podemos ver que las predicciones recursivas (en azul) tienden a capturar mejor fluctuaciones intermedias de los datos, aunque presenta más variabilidad e, indudablemente, es sensible a la acumulación de error conforme avanza el horizonte; mientras que las predicciones directas (en rojo) generan trayectorias más suaves, aunque con un sesgo o crecimiento inicial más abrupto. Como era de esperar, el modelo recursivo genera muy bien movimientos a corto plazo, mientras que el directo capta mejor las tendencias generales.



Figura 13: Predicciones para horizonte H=4

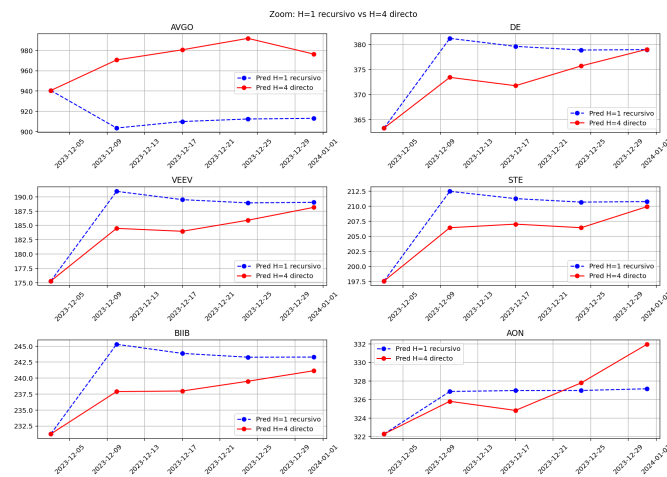


Figura 14: Comparación de predicciones para horizontes H=1 y H=4

5. Conclusiones

En este trabajo se ha desarrollado un algoritmo de Machine Learning para predecir series temporales financieras, aplicando diferentes modelos de aprendizaje automático: XGBoost, LightGBM, LSTM, GRU y Conv1D; con horizontes de predicción de una semana ($H=1$) y un mes ($H=4$). El objetivo del mismo era determinar en qué empresa sería más rentable invertir.

Se realizó un análisis inicial de los datos y un proceso de limpieza de los mismos, garantizando series consistentes, sin valores faltantes, y construyendo datasets supervisados con ventanas deslizantes. Posteriormente, se crearon y entrenaron los modelos, aplicando validación cruzada temporal para evaluar el rendimiento de los modelos evitando fugas de información.

Los resultados revelaron que el modelo convolucional ofreció las mejores métricas, logrando capturar muy bien los patrones locales de las series, seguido de los modelos basados en árboles de decisión (XGboost y LightGBM), mientras que los modelos de arquitecturas recurrentes (LSTM y GRU) mostraron un rendimiento inferior al esperado, probablemente por la complejidad de sus parámetros y la naturaleza de los datos. La incorporación de la técnica de *bagging* permitió incrementar, aún más, el rendimiento de los modelos, especialmente en horizontes temporales cortos.

Como ampliaciones o mejoras futuras de este trabajo, además de la puesta en producción del modelo en casos reales, el seguir ampliando los datos de entrenamiento semanalmente, conforme se vayan produciendo, así como ampliar los datos históricos previos. También se podrían añadir a los modelos variables exógenas, como indicadores macroeconómicos o noticias financieras, para enriquecer el contexto del modelo. También resultaría interesante poder explorar opciones más avanzadas, como transformers, o probar con modelos híbridos, que combinen las fortalezas de las RNN y las CNN.

6. Bibliografía

1. Scikit-learn: <https://scikit-learn.org/stable/>
2. XGBoost: <https://xgboost.readthedocs.io/>
3. LightGBM: <https://lightgbm.readthedocs.io/>
4. TensorFlow Keras: <https://keras.io/>
5. Hyndman & Athanasopoulos, *Forecasting: Principles and Practice*: <https://otexts.com/fpp3/>
6. Machine Learning Mastery, explicación del bagging: <https://machinelearningmastery.com/bagging-and-random-forest-ensemble-algorithms-for-machine-learning/>
7. Validación cruzada en series temporales: https://scikit-learn.org/stable/modules/cross_validation.html#time-series-split

A. Código EDA

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from statsmodels.tsa.seasonal import seasonal_decompose
6 from statsmodels.stats.diagnostic import acorr_ljungbox
7
8 max_nan_ratio = 0.4
9 path_csv = "stock_details_5_years.csv"
10 export_path = "stock_weekly_clean.csv"
11
12 # ----- MAIN -----
13 if __name__ == "__main__":
14     path_csv = "stock_details_5_years.csv"
15
16     df = pd.read_csv(path_csv)
17     df['Date'] = pd.to_datetime(df['Date'], utc=True)
18
19     # EDA preliminar antes del pivotado
20     print("\n=== EDA RAW (datos crudos) ===")
21     print(f"Empresas totales: {df['Company'].nunique()}")
22     print(f"Rango de fechas: {df['Date'].min().date()} a {df['Date'].max().date()}")
23
24     # Conteo por empresa
25     counts = df.groupby("Company")['Date'].count().sort_values(ascending=False)
26     print("\nTop 10 empresas con más registros:")
27     print(counts.head(10))
28     print("\nBottom 10 empresas con menos registros:")
29     print(counts.tail(10))
30
31     # Resample semanal
32     df_pivot = df.pivot(index='Date', columns='Company', values='Close').resample('W').last()
33
34     print("\n=== EDA Pivot (serie temporal por empresa) ===")
35     print(f"Dimensiones: {df_pivot.shape}")
36     print(f"Rango de fechas: {df_pivot.index.min().date()} a {df_pivot.index.max().date()}")
37
38     # Missing por empresa
39     na_ratio = df_pivot.isna().mean().sort_values(ascending=False)
40     na_ratio_row = na_ratio.to_frame().T
41     print("\n--- Porcentaje de missing por empresa (Top 10) ---")
42     print(na_ratio.head(10))
43
44     # Missing por fecha

```

```

45 missing_by_date = df_pivot.isna().sum(axis=1)
46 plt.figure(figsize=(12, 4))
47 missing_by_date.plot()
48 plt.title("Número de valores faltantes por fecha (pre-limpieza)")
49 plt.xlabel("Fecha")
50 plt.ylabel("Num. de missing")
51 plt.show()
52
53 # Empresas eliminadas por exceso de NaN
54 dropped = na_ratio[na_ratio > max_nan_ratio]
55 if not dropped.empty:
56     print(f"\nEmpresas eliminadas (> {max_nan_ratio*100:.0f}% NaN): {
57         list(dropped.index)}")
58     df_pivot = df_pivot.drop(columns=dropped.index)
59
60 # Missing por fecha (tras eliminar empresas malas)
61 missing_by_date = df_pivot.isna().sum(axis=1)
62 plt.figure(figsize=(12, 4))
63 missing_by_date.plot()
64 plt.title("Número de valores faltantes por fecha (tras limpieza)")
65 plt.xlabel("Fecha")
66 plt.ylabel("Num. de missing")
67 plt.show()
68
69 # Imputación: interpolación temporal + ffill + bfill
70 df_pivot = df_pivot.interpolate(method='time').bfill()
71 print(f"Valores missing totales: {df_pivot.isna().sum().sum()}")
72
73 # Estadísticas
74 print("\n--- Estadísticas descriptivas (primeras 10 empresas) ---")
75 descr = df_pivot.describe().T
76 print(descr.head(10))
77 print("\n--- Estadísticas descriptivas (todas las empresas) ---")
78 print(descr)
79
80 # Chequeo de estacionalidad
81 print("\n--- Chequeo de estacionalidad ---")
82 seasonal_strength = {}
83 for company in df_pivot.columns:
84     try:
85         series = df_pivot[company].dropna()
86         decomposition = seasonal_decompose(series, period=52)
87         strength = decomposition.seasonal.var() / (series.var() + 1e
88             -6)
89         seasonal_strength[company] = strength
90     except:
91         seasonal_strength[company] = np.nan
92
93 seasonal_strength_series = pd.Series(seasonal_strength).sort_values(
94     ascending=False)

```

```

93     # Top 20 en fuerza estacional
94     plt.figure(figsize=(12, 5))
95     sns.barplot(x=seasonal_strength_series.index[:20], y=
96     seasonal_strength_series.values[:20], color="steelblue")
97     plt.xticks(rotation=90)
98     plt.ylabel("Fuerza estacionalidad")
99     plt.title("Top 20 empresas por fuerza estacional (periodo=52 semanas)
100     ")
101     plt.show()
102
103     # Descomposición de 3 ejemplos: mayor, menor y mediana fuerza
104     estacional
105     examples = [
106         seasonal_strength_series.dropna().idxmax(),
107         seasonal_strength_series.dropna().idxmin(),
108         seasonal_strength_series.dropna().index[len(
109         seasonal_strength_series)//2]
110     ]
111     print(f"\nEjemplos de descomposición estacional (estacionalidad má
112     xima, mínima y mediana): {examples}")
113     for ex in examples:
114         series = df_pivot[ex].dropna()
115         decomposition = seasonal_decompose(series, period=52)
116         decomposition.plot()
117         plt.show()
118
119         residual = decomposition.resid
120         ljung_box_result = acorr_ljungbox(residual.dropna(), lags=[40],
121         return_df=True)
122         print(f"\nResultado de la prueba de Ljung-Box para la serie {ex}:
123         {ljung_box_result['lb_pvalue'].iloc[0]}")
124
125     # Gráfico comparativo de las 3 series
126     plt.figure(figsize=(12, 5))
127     for ex in examples:
128         plt.plot(df_pivot.index, df_pivot[ex], label=ex)
129     plt.legend()
130     plt.title("Comparación de series: mayor, menor y mediana
131     estacionalidad")
132     plt.grid(True)
133     plt.show()
134
135     # Gráfico de todas las empresas
136     plt.figure(figsize=(12, 5))
137     for company in df_pivot.columns:
138         plt.plot(df_pivot.index, df_pivot[company], label=company)
139     #plt.legend()
140     plt.title("Gráfico de todas las empresas (tras limpieza)")
141     plt.grid(True)
142     plt.show()

```

```
136     # Exportar dataset limpio
137     df_pivot.to_csv(export_path)
138     print(f"\nDataset limpio exportado a: {export_path}")
```

B. Código para H=4

```
1 import numpy as np
2 import pandas as pd
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.metrics import mean_squared_error, r2_score,
  mean_absolute_error, explained_variance_score
5 from sklearn.model_selection import ParameterGrid, TimeSeriesSplit
6 from xgboost import XGBRegressor
7 from lightgbm import LGBMRegressor
8 from tensorflow.keras.models import Sequential
9 from tensorflow.keras.layers import LSTM, GRU, Conv1D, Flatten, Dropout,
  Dense
10 from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
11 from itertools import product
12 from sklearn.base import clone
13 import matplotlib.pyplot as plt
14
15 import warnings
16 warnings.filterwarnings("ignore")
17
18 SEED = 42
19
20 def create_supervised_with_dates(df, W=25, H=1):
21     """
22     Genera un conjunto de datos supervisado a partir de una serie
23     temporal,
24     utilizando ventanas deslizantes y manteniendo las fechas
25     correspondientes.
26
27     Parameters
28     -----
29     df : pandas.DataFrame
30         DataFrame con índices de fechas y columnas de series temporales.
31     W : int, optional (default=25)
32         Longitud de la ventana de entrada (número de observaciones
33         pasadas).
34     H : int, optional (default=1)
35         Horizonte de predicción (número de pasos a predecir).
36
37     Returns
38     -----
39     X : numpy.ndarray
40         Matriz de entradas con las ventanas de tamaño W.
41     y : numpy.ndarray
42         Matriz de salidas con los valores futuros a predecir.
43     dates : numpy.ndarray
44         Fechas correspondientes a cada salida en y.
45     """
46     X, y, dates = [], [], []
```

```

44     for col in df.columns:
45         data = df[col].values
46         idx = df.index
47         for i in range(len(data) - W - H + 1):
48             X.append(data[i:i+W])
49             # H=1 -> sacamos un escalar, pero lo guardamos como vector (n
             ,1) para Keras/consistencia
50             y.append([data[i+W]])
51             dates.append([idx[i+W]]) # fecha objetivo (una semana
             siguiente)
52     return np.array(X), np.array(y), np.array(dates)
53
54 # METRICS
55 def compute_metrics(y_true, y_pred):
56     """
57     Calcula varias métricas de evaluación para modelos de predicción.
58
59     Parameters
60     -----
61     y_true : array-like
62         Valores reales de la serie. Puede tener forma (n,) o (n,1).
63     y_pred : array-like
64         Valores predichos por el modelo. Puede tener forma (n,) o (n,1).
65
66     Returns
67     -----
68     dict
69         Diccionario con las métricas calculadas:
70         - "MSE" : Error cuadrático medio.
71         - "RMSE" : Raíz del error cuadrático medio.
72         - "MAE" : Error absoluto medio.
73         - "R2" : Coeficiente de determinación.
74         - "MAPE" : Error porcentual absoluto medio.
75         - "EVS" : Varianza explicada.
76     """
77     y_true = np.ravel(y_true)
78     y_pred = np.ravel(y_pred)
79     mse = mean_squared_error(y_true, y_pred)
80     rmse = np.sqrt(mse)
81     mae = mean_absolute_error(y_true, y_pred)
82     r2 = r2_score(y_true, y_pred)
83     mape = np.mean(np.abs((y_true - y_pred)/(y_true+1e-6))) * 100
84     evs = explained_variance_score(y_true, y_pred)
85     return {"MSE":mse, "RMSE":rmse, "MAE":mae, "R2":r2, "MAPE":mape, "EVS":evs
86     }
87
88 # DL BUILDERS (H=1)
89 def build_lstm(input_shape, units=16, dropout=0.3, H=1, **kwargs):
90     """
91     Construye un modelo LSTM para predicción de series temporales.

```

```

92     Parameters
93     -----
94     input_shape : tuple
95         Forma de la entrada (timesteps, features).
96     units : int, optional (default=16)
97         Número de unidades en la capa LSTM.
98     dropout : float, optional (default=0.3)
99         Tasa de dropout para regularización.
100     H : int, optional (default=1)
101         Horizonte de predicción (número de pasos a predecir).
102     **kwargs : dict
103         Argumentos adicionales para compatibilidad.
104
105     Returns
106     -----
107     keras.Sequential
108         Modelo LSTM compilado con optimizador Adam y pérdida MSE.
109     """
110     model = Sequential([
111         LSTM(units, input_shape=input_shape),
112         Dropout(dropout),
113         Dense(H)
114     ])
115     model.compile(optimizer='adam', loss='mse')
116     return model
117
118 def build_gru(input_shape, units=16, dropout=0.3, H=1, **kwargs):
119     """
120     Construye un modelo GRU para predicción de series temporales.
121
122     Parameters
123     -----
124     input_shape : tuple
125         Forma de la entrada (timesteps, features).
126     units : int, optional (default=16)
127         Número de unidades en la capa GRU.
128     dropout : float, optional (default=0.3)
129         Tasa de dropout para regularización.
130     H : int, optional (default=1)
131         Horizonte de predicción (número de pasos a predecir).
132     **kwargs : dict
133         Argumentos adicionales para compatibilidad.
134
135     Returns
136     -----
137     keras.Sequential
138         Modelo GRU compilado con optimizador Adam y pérdida MSE.
139     """
140     model = Sequential([
141         GRU(units, input_shape=input_shape),
142         Dropout(dropout),

```

```

143         Dense(H)
144     ])
145     model.compile(optimizer='adam', loss='mse')
146     return model
147
148 def build_conv1d(input_shape, units=16, dropout=0.3, kernel_size=3, H=1,
149 **kwargs):
150     """
151     Construye un modelo Conv1D para predicción de series temporales.
152
153     Parameters
154     -----
155     input_shape : tuple
156         Forma de la entrada (timesteps, features).
157     units : int, optional (default=16)
158         Número de filtros en la capa Conv1D.
159     dropout : float, optional (default=0.3)
160         Tasa de dropout para regularización.
161     kernel_size : int, optional (default=3)
162         Tamaño de la ventana de convolución.
163     H : int, optional (default=1)
164         Horizonte de predicción (número de pasos a predecir).
165     **kwargs : dict
166         Argumentos adicionales para compatibilidad.
167
168     Returns
169     -----
170     keras.Sequential
171         Modelo Conv1D compilado con optimizador Adam y pérdida MSE.
172     """
173     model = Sequential([
174         Conv1D(filters=units, kernel_size=kernel_size, activation='relu',
175               input_shape=input_shape),
176         Flatten(),
177         Dropout(dropout),
178         Dense(H)
179     ])
180     model.compile(optimizer='adam', loss='mse')
181     return model
182
183 # ML CON TIMESERIESSPLIT
184 def tscv_ml(model, X, y, n_splits):
185     """
186     Realiza validación cruzada con TimeSeriesSplit para un modelo de
187     machine learning en series temporales.
188
189     Parameters
190     -----
191     model : estimator
192         Modelo base compatible con scikit-learn.
193     X : numpy.ndarray

```

```

192     Conjunto de entradas de forma (n_muestras, ventana, features).
193 y : numpy.ndarray
194     Conjunto de salidas de forma (n_muestras,) o (n_muestras, 1).
195 n_splits : int
196     Número de divisiones para TimeSeriesSplit.
197
198 Returns
199 -----
200 dict
201     Diccionario con el promedio de métricas en todos los splits:
202     - "MSE"      : Error cuadrático medio.
203     - "RMSE"     : Raíz del error cuadrático medio.
204     - "MAE"      : Error absoluto medio.
205     - "R2"       : Coeficiente de determinación.
206     - "MAPE"     : Error porcentual absoluto medio.
207     - "EVS"      : Varianza explicada.
208 """
209 y = np.ravel(y)
210 tscv = TimeSeriesSplit(n_splits=n_splits)
211 metrics_list = []
212 X_flat = X.reshape((X.shape[0], -1))
213 for train_idx, test_idx in tscv.split(X_flat):
214     X_train, X_test = X_flat[train_idx], X_flat[test_idx]
215     y_train, y_test = y[train_idx], y[test_idx]
216     scaler = StandardScaler()
217     X_train_scaled = scaler.fit_transform(X_train)
218     X_test_scaled = scaler.transform(X_test)
219     model_fold = clone(model)
220     model_fold.fit(X_train_scaled, y_train)
221     y_pred = model_fold.predict(X_test_scaled)
222     metrics_list.append(compute_metrics(y_test, y_pred))
223 return {k: np.mean([m[k] for m in metrics_list]) for k in
224         metrics_list[0]}
225
226 # ----- TSCV: DL -----
227 def tscv_dl(model_builder, X, y, n_splits, units, dropout, epochs,
228             batch_size, kernel_size=3):
229     """
230     Realiza validación cruzada con TimeSeriesSplit para un modelo de
231     deep learning en series temporales.
232
233     Parameters
234     -----
235     model_builder : callable
236         Función constructora del modelo (ej. build_lstm, build_gru,
237         build_conv1d).
238     X : numpy.ndarray
239         Conjunto de entradas de forma (n_muestras, ventana, features).
240     y : numpy.ndarray
241         Conjunto de salidas de forma (n_muestras,) o (n_muestras, 1).
242     n_splits : int

```

```

240     Número de divisiones para TimeSeriesSplit.
241     units : int
242     Número de unidades o filtros en la capa principal.
243     dropout : float
244     Tasa de dropout para regularización.
245     epochs : int
246     Número de épocas de entrenamiento.
247     batch_size : int
248     Tamaño del lote en el entrenamiento.
249     kernel_size : int, optional (default=3)
250     Tamaño de kernel en la capa Conv1D (si aplica).
251
252     Returns
253     -----
254     dict
255     Diccionario con el promedio de métricas en todos los splits:
256     - "MSE" : Error cuadrático medio.
257     - "RMSE" : Raíz del error cuadrático medio.
258     - "MAE" : Error absoluto medio.
259     - "R2" : Coeficiente de determinación.
260     - "MAPE" : Error porcentual absoluto medio.
261     - "EVS" : Varianza explicada.
262
263     """
264     tscv = TimeSeriesSplit(n_splits=n_splits)
265     metrics_list = []
266     X_seq = X.reshape((X.shape[0], X.shape[1], 1))
267     y = y.reshape(-1, 1)
268
269     for train_idx, test_idx in tscv.split(X_seq):
270         X_train, X_test = X_seq[train_idx], X_seq[test_idx]
271         y_train, y_test = y[train_idx], y[test_idx]
272
273         # Estandarizamos X por ventana (aplanamos y re-formamos)
274         scaler = StandardScaler()
275         X_train_2d = X_train.reshape((X_train.shape[0], -1))
276         X_test_2d = X_test.reshape((X_test.shape[0], -1))
277         X_train_scaled = scaler.fit_transform(X_train_2d).reshape(X_train
278         .shape)
279         X_test_scaled = scaler.transform(X_test_2d).reshape(X_test.shape)
280
281         model = model_builder(input_shape=(X_train.shape[1],1),
282                                units=units, dropout=dropout, H=1,
283                                kernel_size=kernel_size)
284         es = EarlyStopping(monitor='val_loss', patience=2,
285                             restore_best_weights=True)
286         lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience
287                                =1)
288         model.fit(X_train_scaled, y_train, validation_data=(X_test_scaled
289         , y_test),
290                 epochs=epochs, batch_size=batch_size, verbose=0,
291                 callbacks=[es, lr])

```

```

285         y_pred = model.predict(X_test_scaled, verbose=0)
286         metrics_list.append(compute_metrics(y_test, y_pred))
287
288     return {k: np.mean([m[k] for m in metrics_list]) for k in
289             metrics_list[0]}
290
291 # GRAFICOS
292 def plot_learning_curves(histories, title="Curvas de entrenamiento/
293 validación"):
294     """
295     Grafica las curvas de pérdida (loss) de entrenamiento y validación
296     a partir de objetos History de Keras.
297
298     Parameters
299     -----
300     histories : keras.callbacks.History or dict
301         Puede ser:
302         - Un único objeto History (para un solo modelo).
303         - Un diccionario {nombre: history} para graficar varios modelos
304         juntos.
305     title : str, optional
306         Título del gráfico. Por defecto "Curvas de entrenamiento/validaci
307         ón".
308
309     Returns
310     -----
311     None
312
313     Muestra el gráfico en pantalla.
314     """
315     plt.figure(figsize=(8,6))
316
317     if isinstance(histories, dict):
318         # Varios modelos
319         for name, hist in histories.items():
320             plt.plot(hist.history['loss'], label=f'{name} - Entrenamiento
321 ')
322             if 'val_loss' in hist.history:
323                 plt.plot(hist.history['val_loss'], label=f'{name} -
324 Validación')
325
326     else:
327         # Un solo modelo
328         hist = histories
329         plt.plot(hist.history['loss'], label='Entrenamiento')
330         if 'val_loss' in hist.history:
331             plt.plot(hist.history['val_loss'], label='Validación')
332
333     plt.xlabel("Epocas")
334     plt.ylabel("Loss (MSE)")
335     plt.title(title)
336     plt.legend()
337     plt.grid(True)

```

```

330     plt.show()
331
332     # MAIN
333     if __name__=="__main__":
334         path_csv = "stock_weekly_clean.csv"
335         W, H = 25, 1
336         sub_sample_ratio = 0.4
337         np.random.seed(SEED)
338
339         df_weekly = pd.read_csv(path_csv, index_col=0, parse_dates=True)
340         df_weekly.index = pd.to_datetime(df_weekly.index, utc=True)
341         sampled_cols = np.random.choice(df_weekly.columns, int(len(df_weekly.
342             columns)*sub_sample_ratio), replace=False)
343         df_sub = df_weekly[sampled_cols]
344
345         X_sub, y_sub, _ = create_supervised_with_dates(df_sub, W=W, H=H)
346         X_sub = np.nan_to_num(X_sub)
347         y_sub = np.nan_to_num(y_sub) # (n,1)
348
349         # HYPERPARAMETERS
350         xgb_params = {"n_estimators": [100, 200], "max_depth": [3, 5], "
351             learning_rate": [0.01, 0.05]}
352         lgb_params = {"n_estimators": [100, 200], "max_depth": [5, 10], "
353             learning_rate": [0.01, 0.05]}
354         dl_params = {'units': [16, 32, 64], 'dropout': [0.2, 0.3]}
355         cid_params = {'units': [16, 32, 64], 'dropout': [0.2, 0.3], 'kernel_size'
356             : [3, 5]}
357
358         best_models = {}
359         tuning_results = {}
360         results_sub = {}
361
362         # ML MODELS (tuning con TSCV)
363         ml_models = {"XGB": XGBRegressor, "LGBM": LGBMRegressor}
364         ml_param_grids = {"XGB": xgb_params, "LGBM": lgb_params}
365
366         for name, ModelClass in ml_models.items():
367             best_score, best_model = -np.inf, None
368             for params in ParameterGrid(ml_param_grids[name]):
369                 print(f"{name} con {params}")
370                 if name=="XGB":
371                     model = ModelClass(random_state=SEED, verbosity=0, **
372                         params)
373                 else:
374                     model = ModelClass(random_state=SEED, verbose=-1, **
375                         params)
376                 metrics = tscv_ml(model, X_sub, y_sub, n_splits=5)
377                 tuning_results.setdefault(name, []).append({'params': params,
378                     'metrics': metrics})
379                 if metrics['R2'] > best_score:
380                     best_score = metrics['R2']

```

```

374         best_model = model
375         # Guardamos mejores
376         results_sub[name] = tscv_ml(best_model, X_sub, y_sub, n_splits=5)
377         best_models[name] = best_model
378
379         # DL MODELS (tuning con TSCV)
380         dl_builders = {"LSTM": build_lstm, "GRU": build_gru, "Conv1D":
381         build_conv1d}
382
383         dl_param_grids = {"LSTM": dl_params, "GRU": dl_params, "Conv1D":
384         cid_params}
385
386         for name, builder in dl_builders.items():
387             best_score, best_config, best_metrics = -np.inf, None, None
388             param_grid = dl_param_grids[name]
389
390             if name == "Conv1D":
391                 combos = product(param_grid['units'], param_grid['dropout'],
392                 param_grid['kernel_size'])
393             else:
394                 combos = product(param_grid['units'], param_grid['dropout'])
395
396             for combo in combos:
397                 print(f"{name} con {combo}")
398                 if name == "Conv1D":
399                     units, dropout, kernel_size = combo
400                     metrics = tscv_dl(builder, X_sub, y_sub, n_splits=3,
401                     units=units, dropout=dropout, epochs
402                     =10, batch_size=32,
403                     kernel_size=kernel_size)
404                     config = {'units': units, 'dropout': dropout, '
405                     kernel_size': kernel_size}
406                 else:
407                     units, dropout = combo
408                     metrics = tscv_dl(builder, X_sub, y_sub, n_splits=3,
409                     units=units, dropout=dropout, epochs
410                     =10, batch_size=32)
411                     config = {'units': units, 'dropout': dropout}
412
413                 tuning_results.setdefault(name, []).append({'params': config,
414                 'metrics': metrics})
415
416                 if metrics['R2'] > best_score:
417                     best_score = metrics['R2']
418                     best_config = config
419                     best_metrics = metrics
420
421         # Guardamos todas las métricas del mejor modelo + su config
422         results_sub[name] = best_metrics
423         results_sub[name]['config'] = best_config
424
425         # Guardamos config para reentrenar más adelante

```

```

418         best_models[name] = best_config
419
420     print("\n=== Métricas de los mejores modelos (submuestra) ===")
421     pd.set_option("display.max_rows", None)
422     pd.set_option("display.max_columns", None)
423     pd.set_option("display.width", None)
424     pd.set_option("display.max_colwidth", None)
425     df_results = pd.DataFrame(results_sub).T
426     print(df_results.sort_values("R2", ascending=False))
427
428     # SELECT BEST MODEL
429     best_r2 = -np.inf
430     best_model_name = None
431     for name, m in best_models.items():
432         if name in ["XGB", "LGBM"]:
433             r2 = tscv_ml(m, X_sub, y_sub, n_splits=5)['R2']
434         else:
435             r2 = tscv_dl(dl_builders[name], X_sub, y_sub, n_splits=3, **m
436                          , epochs=10, batch_size=32)['R2']
437         if r2 > best_r2:
438             best_r2 = r2
439             best_model_name = name
440     print(f"Mejor modelo por R2: {best_model_name} (R2={best_r2:.4f})")
441
442     # PREPARAR DATOS COMPLETOS
443     X_full, y_full, dates_full = create_supervised_with_dates(df_sub, W=W
444     , H=H)
445     X_full = np.nan_to_num(X_full)
446     y_full = np.nan_to_num(y_full) # (n,1)
447     scaler_full = StandardScaler()
448     X_full_scaled = scaler_full.fit_transform(X_full)
449
450     # BAGGING GLOBAL (último fold, validación)
451     print("=== Bagging (validación, último fold) ===")
452     top_models = ["LGBM", "Conv1D"]
453     n_bags = 5
454
455     tscv = TimeSeriesSplit(n_splits=5)
456     train_idx, test_idx = list(tscv.split(X_full_scaled))[-1]
457     X_train, X_test = X_full_scaled[train_idx], X_full_scaled[test_idx]
458     y_train, y_test = y_full[train_idx], y_full[test_idx] # (n,1)
459
460     bagging_preds = np.zeros(y_test.shape[0], dtype=float)
461     histories = {}
462
463     for name in top_models:
464         fold_preds = np.zeros(y_test.shape[0], dtype=float)
465         for _ in range(n_bags):
466             if name in ["XGB", "LGBM"]:
467                 base = best_models[name] # es un modelo ya configurado
468                 model = clone(base)

```

```

467         model.fit(X_train, np.ravel(y_train))
468         fold_preds += model.predict(X_test)
469     else:
470         # DL
471         config = best_models[name]
472         X_train_seq = X_train.reshape((X_train.shape[0], W, 1))
473         X_test_seq = X_test.reshape((X_test.shape[0], W, 1))
474         model = dl_builders[name](input_shape=(W,1), H=1, **
475                                   config)
476         es = EarlyStopping(monitor='val_loss', patience=3,
477                           restore_best_weights=True)
478         lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5,
479                               patience=2)
480         history = model.fit(X_train_seq, y_train, validation_data
481                             =(X_test_seq, y_test),
482                               epochs=20, batch_size=32, verbose=0, callbacks
483                               =[es, lr])
484         histories[name] = history
485         fold_preds += model.predict(X_test_seq, verbose=0).
486         flatten()
487     fold_preds /= n_bags
488     bagging_preds += fold_preds
489
490 bagging_preds /= len(top_models)
491 metrics = compute_metrics(y_test, bagging_preds)
492 plot_learning_curves(histories, title="Curvas conjuntas de
493 entrenamiento/validación (DL)")
494 print("Métricas Bagging (H=1):", {k: round(v,4) for k,v in metrics.
495 items()})
496
497 # REENTRENAR EN TODOS LOS DATOS
498 print("\n=== Reentrenamiento en todos los datos para predicción futura ==="
499 )
500 trained_models = {}
501 for name in top_models:
502     if name in ["XGB", "LGBM"]:
503         base = best_models[name]
504         model = clone(base)
505         model.fit(X_full_scaled, np.ravel(y_full))
506     else:
507         config = best_models[name]
508         X_full_seq = X_full_scaled.reshape((X_full_scaled.shape[0], W
509 , 1))
510         model = dl_builders[name](input_shape=(W,1), H=1, **config)
511         es = EarlyStopping(monitor='val_loss', patience=3,
512                           restore_best_weights=True)
513         lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5,
514                               patience=2)
515         model.fit(X_full_seq, y_full, epochs=20, batch_size=32,
516                   verbose=0, callbacks=[es, lr])

```

```

505         trained_models[name] = model
506
507     # PREDICCIÓN FUTURA
508     print("\n=== Predicción futura (última ventana de cada empresa, H=1) ===")
509     n_companies = df_sub.shape[1]
510     y_pred_last = np.zeros(n_companies)
511
512     for i, company in enumerate(df_sub.columns):
513         series = df_sub[company].values
514         last_window = series[-W:].reshape(1, -1)
515         last_window_scaled = scaler_full.transform(last_window)
516
517         preds_models = []
518         for name, model in trained_models.items():
519             if name in ["XGB", "LGBM"]:
520                 preds_models.append(float(model.predict(
521                     last_window_scaled)[0]))
522             else:
523                 last_seq = last_window_scaled.reshape((1, W, 1))
524                 preds_models.append(float(model.predict(last_seq, verbose
525                     =0).flatten()[0]))
526         y_pred_last[i] = np.mean(preds_models) # bagging entre modelos
527
528     # RESULTADOS
529     last_values = df_sub.iloc[-1].values
530     diff_abs = y_pred_last - last_values
531     diff_pct = 100 * diff_abs / (last_values + 1e-6)
532     top6_idx = np.argsort(diff_abs)[-6:][::-1]
533
534     print("\nTop 6 empresas con mayor crecimiento previsto (H=1):")
535     for idx in top6_idx:
536         print(f"{df_sub.columns[idx]:<4}: Actual={last_values[idx]:>8.2f} | "
537             f"Pred={y_pred_last[idx]:>8.2f} | "
538             f"DifAbs={diff_abs[idx]:>7.2f} | Dif%={diff_pct[idx]:>6.2f}%")
539
540     # GRAFICO
541     fig, axes = plt.subplots(3,2, figsize=(14,10))
542     axes = axes.flatten()
543     last_date = df_sub.index[-1]
544     next_date = last_date + pd.Timedelta(weeks=1)
545
546     for j, idx in enumerate(top6_idx):
547         company = df_sub.columns[idx]
548         axes[j].plot(df_sub.index, df_sub[company].values, label="Histórico", color="black")
549         axes[j].scatter([next_date], [y_pred_last[idx]], color="red",
550             label="Predicción (H=1)", s=70, marker="x")
551         axes[j].plot([last_date, next_date], [last_values[idx],
552             y_pred_last[idx]], color="red", linestyle="--")
553         axes[j].set_title(company)

```

```
549         axes[j].legend()  
550         axes[j].grid(True)  
551  
552     plt.tight_layout()  
553     plt.show()
```

C. Código para H=1

```

1 import numpy as np
2 import pandas as pd
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.metrics import mean_squared_error, r2_score,
  mean_absolute_error, explained_variance_score
5 from sklearn.multioutput import MultiOutputRegressor
6 from sklearn.model_selection import ParameterGrid, TimeSeriesSplit
7 from xgboost import XGBRegressor
8 from lightgbm import LGBMRegressor
9 from tensorflow.keras.models import Sequential
10 from tensorflow.keras.layers import LSTM, GRU, Conv1D, Flatten, Dropout,
  Dense
11 from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
12 from itertools import product
13 from sklearn.base import clone
14 import matplotlib.pyplot as plt
15
16 import warnings
17 warnings.filterwarnings("ignore")
18
19 SEED = 42
20
21 def create_supervised_with_dates(df, W=25, H=4):
22     """
23     Genera un conjunto de datos supervisado a partir de una serie
24     temporal,
25     utilizando ventanas deslizantes y manteniendo las fechas
26     correspondientes.
27
28     Parameters
29     -----
30     df : pandas.DataFrame
31         DataFrame con índices de fechas y columnas de series temporales.
32     W : int, optional (default=25)
33         Longitud de la ventana de entrada (número de observaciones
34         pasadas).
35     H : int, optional (default=1)
36         Horizonte de predicción (número de pasos a predecir).
37
38     Returns
39     -----
40     X : numpy.ndarray
41         Matriz de entradas con las ventanas de tamaño W.
42     y : numpy.ndarray
43         Matriz de salidas con los valores futuros a predecir.
44     dates : numpy.ndarray
45         Fechas correspondientes a cada salida en y.
46     """

```

```

44     X, y, dates = [], [], []
45     for col in df.columns:
46         data = df[col].values
47         idx = df.index
48         for i in range(len(data)-W-H+1):
49             X.append(data[i:i+W])
50             y.append(data[i+W:i+W+H])
51             dates.append(idx[i+W:i+W+H])
52     return np.array(X), np.array(y), np.array(dates)
53
54 # METRICS
55 def compute_metrics(y_true, y_pred):
56     """
57     Calcula varias métricas de evaluación para modelos de predicción.
58
59     Parameters
60     -----
61     y_true : array-like
62         Valores reales de la serie.
63     y_pred : array-like
64         Valores predichos por el modelo.
65
66     Returns
67     -----
68     dict
69         Diccionario con las métricas calculadas:
70         - "MSE" : Error cuadrático medio.
71         - "RMSE" : Raíz del error cuadrático medio.
72         - "MAE" : Error absoluto medio.
73         - "R2" : Coeficiente de determinación.
74         - "MAPE" : Error porcentual absoluto medio.
75         - "EVS" : Varianza explicada.
76     """
77     mse = mean_squared_error(y_true, y_pred)
78     rmse = np.sqrt(mse)
79     mae = mean_absolute_error(y_true, y_pred)
80     r2 = r2_score(y_true, y_pred)
81     mape = np.mean(np.abs((y_true - y_pred)/(y_true+1e-6))) * 100
82     da = np.mean((y_true>0) == (y_pred>0))
83     smape = 100*np.mean(2*np.abs(y_pred-y_true)/(np.abs(y_true)+np.abs(y_pred)+1e-6))
84     evs = explained_variance_score(y_true, y_pred)
85     return {"MSE":mse, "RMSE":rmse, "MAE":mae, "R2":r2, "MAPE":mape, "EVS":evs}
86
87 # DL MODELS
88 def build_lstm(input_shape, units=16, dropout=0.3, H=4, **kwargs):
89     """
90     Construye un modelo LSTM para predicción de series temporales.
91
92     Parameters

```

```

93     -----
94     input_shape : tuple
95         Forma de la entrada (timesteps, features).
96     units : int, optional (default=16)
97         Número de unidades en la capa LSTM.
98     dropout : float, optional (default=0.3)
99         Tasa de dropout para regularización.
100    H : int, optional (default=4)
101        Horizonte de predicción (número de pasos a predecir).
102    **kwargs : dict
103        Argumentos adicionales para compatibilidad.
104
105    Returns
106    -----
107    keras.Sequential
108        Modelo LSTM compilado con optimizador Adam y pérdida MSE.
109    """
110    model = Sequential([LSTM(units, input_shape=input_shape),
111                        Dropout(dropout),
112                        Dense(H)])
113    model.compile(optimizer='adam', loss='mse')
114    return model
115
116 def build_gru(input_shape, units=16, dropout=0.3, H=4, **kwargs):
117     """
118     Construye un modelo GRU para predicción de series temporales.
119
120     Parameters
121     -----
122     input_shape : tuple
123         Forma de la entrada (timesteps, features).
124     units : int, optional (default=16)
125         Número de unidades en la capa GRU.
126     dropout : float, optional (default=0.3)
127         Tasa de dropout para regularización.
128     H : int, optional (default=4)
129         Horizonte de predicción (número de pasos a predecir).
130     **kwargs : dict
131         Argumentos adicionales para compatibilidad.
132
133     Returns
134     -----
135     keras.Sequential
136         Modelo GRU compilado con optimizador Adam y pérdida MSE.
137     """
138     model = Sequential([GRU(units, input_shape=input_shape),
139                        Dropout(dropout),
140                        Dense(H)])
141     model.compile(optimizer='adam', loss='mse')
142     return model
143

```

```

144 def build_conv1d(input_shape, units=16, dropout=0.3, kernel_size=3, H=4,
145 **kwargs):
146     """
147     Construye un modelo Conv1D para predicción de series temporales.
148
149     Parameters
150     -----
151     input_shape : tuple
152         Forma de la entrada (timesteps, features).
153     units : int, optional (default=16)
154         Número de filtros en la capa Conv1D.
155     dropout : float, optional (default=0.3)
156         Tasa de dropout para regularización.
157     kernel_size : int, optional (default=3)
158         Tamaño de la ventana de convolución.
159     H : int, optional (default=4)
160         Horizonte de predicción (número de pasos a predecir).
161     **kwargs : dict
162         Argumentos adicionales para compatibilidad.
163
164     Returns
165     -----
166     keras.Sequential
167         Modelo Conv1D compilado con optimizador Adam y pérdida MSE.
168     """
169     model = Sequential([Conv1D(filters=units, kernel_size=kernel_size,
170                               activation='relu', input_shape=input_shape),
171                        Flatten(),
172                        Dropout(dropout),
173                        Dense(H)])
174     model.compile(optimizer='adam', loss='mse')
175     return model
176
177 # ML con TimeSeriesSplit
178 def tscv_ml(model, X, y, n_splits):
179     """
180     Realiza validación cruzada para modelos de machine learning en series
181     temporales.
182
183     Parameters
184     -----
185     model : estimator
186         Modelo base compatible con scikit-learn.
187     X : numpy.ndarray
188         Conjunto de entradas, de forma (n_muestras, ventana, features).
189     y : numpy.ndarray
190         Conjunto de salidas, de forma (n_muestras, horizonte).
191     n_splits : int
192         Número de divisiones para TimeSeriesSplit.
193
194     Returns

```

```

192 -----
193 dict
194     Diccionario con el promedio de métricas en todos los splits:
195     - "MSE"      : Error cuadrático medio.
196     - "RMSE"     : Raíz del error cuadrático medio.
197     - "MAE"      : Error absoluto medio.
198     - "R2"       : Coeficiente de determinación.
199     - "MAPE"     : Error porcentual absoluto medio.
200     - "EVS"      : Varianza explicada.
201     """
202     tscv = TimeSeriesSplit(n_splits=n_splits)
203     metrics_list = []
204     X_flat = X.reshape((X.shape[0], -1))
205     for train_idx, test_idx in tscv.split(X_flat):
206         X_train, X_test = X_flat[train_idx], X_flat[test_idx]
207         y_train, y_test = y[train_idx], y[test_idx]
208         scaler = StandardScaler()
209         X_train_scaled = scaler.fit_transform(X_train)
210         X_test_scaled = scaler.transform(X_test)
211         model_fold = MultiOutputRegressor(clone(model)) if y.shape[1]>1
212         else clone(model)
213         model_fold.fit(X_train_scaled, y_train)
214         y_pred = model_fold.predict(X_test_scaled)
215         metrics_list.append(compute_metrics(y_test, y_pred))
216     return {k: np.mean([m[k] for m in metrics_list]) for k in
217             metrics_list[0]}
218
219 # DL con TimeSeriesSplit
220 def tscv_dl(model_builder, X, y, n_splits, units, dropout, epochs,
221             batch_size, kernel_size=3):
222     """
223     Realiza validación cruzada para modelos de deep learning en series
224     temporales.
225
226     Parameters
227     -----
228     model_builder : callable
229         Función constructora del modelo (ej. build_lstm, build_gru,
230         build_conv1d).
231     X : numpy.ndarray
232         Conjunto de entradas, de forma (n_muestras, ventana, features).
233     y : numpy.ndarray
234         Conjunto de salidas, de forma (n_muestras, horizonte).
235     n_splits : int
236         Número de divisiones para TimeSeriesSplit.
237     units : int
238         Número de unidades/filtros en la capa principal.
239     dropout : float
240         Tasa de dropout para regularización.
241     epochs : int
242         Número de épocas de entrenamiento.

```

```

238     batch_size : int
239         Tamaño del lote en el entrenamiento.
240     kernel_size : int, optional (default=3)
241         Tamaño de kernel en la capa Conv1D (si aplica).
242
243     Returns
244     -----
245     dict
246         Diccionario con el promedio de métricas en todos los splits:
247     - "MSE"      : Error cuadrático medio.
248     - "RMSE"     : Raíz del error cuadrático medio.
249     - "MAE"      : Error absoluto medio.
250     - "R2"       : Coeficiente de determinación.
251     - "MAPE"     : Error porcentual absoluto medio.
252     - "EVS"      : Varianza explicada.
253     """
254     tscv = TimeSeriesSplit(n_splits=n_splits)
255     metrics_list = []
256     X_seq = X.reshape((X.shape[0], X.shape[1], 1))
257
258     for train_idx, test_idx in tscv.split(X_seq):
259         X_train, X_test = X_seq[train_idx], X_seq[test_idx]
260         y_train, y_test = y[train_idx], y[test_idx]
261         scaler = StandardScaler()
262         X_train_2d = X_train.reshape((X_train.shape[0], -1))
263         X_test_2d = X_test.reshape((X_test.shape[0], -1))
264         X_train_scaled = scaler.fit_transform(X_train_2d).reshape(X_train
265         .shape)
266         X_test_scaled = scaler.transform(X_test_2d).reshape(X_test.shape)
267
268         model = model_builder(input_shape=(X_train.shape[1], 1),
269                               units=units, dropout=dropout, H=y_train.
270                               shape[1], kernel_size=kernel_size)
271         es = EarlyStopping(monitor='val_loss', patience=2,
272                             restore_best_weights=True)
273         lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience
274                                =1)
275         model.fit(X_train_scaled, y_train, validation_data=(X_test_scaled
276         , y_test),
277                 epochs=epochs, batch_size=batch_size, verbose=0,
278                 callbacks=[es, lr])
279         y_pred = model.predict(X_test_scaled, verbose=0)
280         metrics_list.append(compute_metrics(y_test, y_pred))
281
282     return {k: np.mean([m[k] for m in metrics_list]) for k in
283             metrics_list[0]}
284
285 # GRAFICOS
286 def plot_learning_curves(histories, title="Curvas de entrenamiento/
287 validación"):
288     """

```

```

281 Grafica las curvas de pérdida (loss) en entrenamiento y validación
282 a partir de objetos History de Keras.
283
284 Parameters
285 -----
286 histories : keras.callbacks.History or dict
287     Puede ser:
288     - Un único objeto History (para un solo modelo).
289     - Un diccionario {nombre: history} para graficar varios modelos
290       juntos.
291 title : str, optional
292     Título del gráfico.
293
294 Returns
295 -----
296 None
297     Muestra el gráfico en pantalla.
298 """
299 plt.figure(figsize=(8,6))
300
301 if isinstance(histories, dict):
302     # Varios modelos
303     for name, hist in histories.items():
304         plt.plot(hist.history['loss'], label=f'{name} - Entrenamiento')
305         if 'val_loss' in hist.history:
306             plt.plot(hist.history['val_loss'], label=f'{name} - Validación')
307 else:
308     # Un solo modelo
309     hist = histories
310     plt.plot(hist.history['loss'], label='Entrenamiento')
311     if 'val_loss' in hist.history:
312         plt.plot(hist.history['val_loss'], label='Validación')
313
314 plt.xlabel("Epocas")
315 plt.ylabel("Loss (MSE)")
316 plt.title(title)
317 plt.legend()
318 plt.grid(True)
319 plt.show()
320
321 if __name__=="__main__":
322     path_csv = "stock_weekly_clean.csv"
323     W,H = 25,4
324     sub_sample_ratio = 0.4
325     np.random.seed(SEED)
326
327     df_weekly = pd.read_csv(path_csv, index_col=0, parse_dates=True)
328     df_weekly.index = pd.to_datetime(df_weekly.index, utc=True)

```

```

329     sampled_cols = np.random.choice(df_weekly.columns, int(len(df_weekly.
330     columns)*sub_sample_ratio), replace=False)
331     df_sub = df_weekly[sampled_cols]
332
333     X_sub, y_sub, _ = create_supervised_with_dates(df_sub, W=W, H=H)
334     X_sub = np.nan_to_num(X_sub)
335     y_sub = np.nan_to_num(y_sub)
336
337     # HYPERPARAMETERS
338     xgb_params = {"n_estimators": [100, 200], "max_depth": [3, 5], "
339     learning_rate": [0.01, 0.05]}
340     lgb_params = {"n_estimators": [100, 200], "max_depth": [5, 10], "
341     learning_rate": [0.01, 0.05]}
342     dl_params = {'units': [16, 32, 64], 'dropout': [0.2, 0.3]}
343     cid_params = {'units': [16, 32, 64], 'dropout': [0.2, 0.3], 'kernel_size'
344     : [3, 5]}
345
346     best_models = {}
347     tuning_results = {}
348     results_sub = {}
349
350     # ML MODELS
351     ml_models = {"XGB": XGBRegressor, "LGBM": LGBMRegressor}
352     ml_param_grids = {"XGB": xgb_params, "LGBM": lgb_params}
353
354     for name, ModelClass in ml_models.items():
355         best_score, best_model = -np.inf, None
356         for params in ParameterGrid(ml_param_grids[name]):
357             print(f"{name} con {params}")
358             if name=="XGB":
359                 model = ModelClass(random_state=SEED, verbosity=0, **
360                 params)
361             else:
362                 model = ModelClass(random_state=SEED, verbose=-1, **
363                 params)
364             metrics = tscv_ml(model, X_sub, y_sub, n_splits=5)
365             tuning_results.setdefault(name, []).append({'params': params,
366             'metrics': metrics})
367             if metrics['R2'] > best_score:
368                 best_score = metrics['R2']
369                 best_model = model
370             results_sub[name] = tscv_ml(best_model, X_sub, y_sub, n_splits=5)
371             best_models[name] = best_model
372
373     # DL MODELS
374     dl_models = {"LSTM": build_lstm, "GRU": build_gru, "Conv1D":
375     build_conv1d}
376     dl_param_grids = {"LSTM": dl_params, "GRU": dl_params, "Conv1D":
377     cid_params}
378
379     for name, builder in dl_models.items():

```

```

371     best_score, best_config, best_metrics = -np.inf, None, None
372     param_grid = dl_param_grids[name]
373     combos = product(*(param_grid.values())) if name!="Conv1D" else
product(param_grid['units'], param_grid['dropout'], param_grid['
kernel_size'])
374     for combo in combos:
375         print(f"{name} con {combo}")
376         if name=="Conv1D":
377             units, dropout, kernel_size = combo
378             metrics = tscv_dl(builder, X_sub, y_sub, n_splits=3,
units=units, dropout=dropout,
379                             kernel_size=kernel_size, epochs=10,
batch_size=32)
380             config = {'units': units, 'dropout': dropout, '
kernel_size': kernel_size}
381         else:
382             units, dropout = combo
383             metrics = tscv_dl(builder, X_sub, y_sub, n_splits=3,
units=units, dropout=dropout,
384                             epochs=10, batch_size=32, kernel_size
=3)
385             config = {'units': units, 'dropout': dropout}
386
387         tuning_results.setdefault(name, []).append({'params': config,
'metrics': metrics})
388
389         if metrics['R2'] > best_score:
390             best_score = metrics['R2']
391             best_config = config
392             best_metrics = metrics
393
394         # Guardamos todas las métricas del mejor modelo
395         results_sub[name] = {**best_metrics, 'config': best_config}
396         best_models[name] = best_config
397
398     # SELECT BEST MODEL
399     best_r2 = -np.inf
400     best_model_name = None
401     for name, m in best_models.items():
402         if name in ["XGB", "LGBM"]:
403             r2 = tscv_ml(m, X_sub, y_sub, n_splits=5)['R2']
404         else:
405             r2 = tscv_dl(dl_models[name], X_sub, y_sub, n_splits=3, **m,
epochs=10, batch_size=32)['R2']
406         if r2 > best_r2:
407             best_r2 = r2
408             best_model_name = name
409
410     print("\n=== Métricas de los mejores modelos (submuestra) ===")
411     pd.set_option("display.max_rows", None)
412     pd.set_option("display.max_columns", None)

```

```

413 pd.set_option("display.width", None)
414 pd.set_option("display.max_colwidth", None)
415 print(pd.DataFrame(results_sub).T)
416
417 top_models = ["LGBM", "Conv1D"]
418
419 # BAGGING GLOBAL
420 print("=== Bagging simple por empresa ===")
421
422 n_bags = 5
423
424 # Preparamos datos completos supervisados
425 X_full, y_full, dates_full = create_supervised_with_dates(df_sub, W=W
426 , H=H)
427 X_full = np.nan_to_num(X_full)
428 y_full = np.nan_to_num(y_full)
429
430 # Escalamos
431 scaler_full = StandardScaler()
432 X_full_scaled = scaler_full.fit_transform(X_full)
433
434 # Ultimo fold train/test
435 tscv = TimeSeriesSplit(n_splits=5)
436 train_idx, test_idx = list(tscv.split(X_full_scaled))[-1]
437 X_train, X_test = X_full_scaled[train_idx], X_full_scaled[test_idx]
438 y_train, y_test = y_full[train_idx], y_full[test_idx]
439
440 # Bagging sobre todos los modelos
441 bagging_preds = np.zeros_like(y_test, dtype=float)
442 histories = {}
443
444 for name in top_models:
445     preds_bag = np.zeros_like(y_test, dtype=float)
446     for _ in range(n_bags):
447         if name in ["XGB", "LGBM"]:
448             model = MultiOutputRegressor(clone(best_models[name])) if
449                 y_train.shape[1] > 1 else clone(best_models[name])
450             model.fit(X_train, y_train)
451             preds_bag += model.predict(X_test)
452         else: # Conv1D
453             config = best_models[name]
454             X_train_seq = X_train.reshape((X_train.shape[0], W, 1))
455             X_test_seq = X_test.reshape((X_test.shape[0], W, 1))
456             model = dl_models[name](input_shape=(W, 1), H=y_train.
457                 shape[1], **config)
458             es = EarlyStopping(monitor='val_loss', patience=3,
459                 restore_best_weights=True)
460             lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5,
461                 patience=2)
462             history = model.fit(X_train_seq, y_train, validation_data
463                 =(X_test_seq, y_test),

```

```

458         epochs=20, batch_size=32, verbose=0, callbacks
459         =[es, lr])
460         histories[name] = history
461         preds_bag += model.predict(X_test_seq, verbose=0)
462     preds_bag /= n_bags
463     bagging_preds += preds_bag
464
465     bagging_preds /= len(top_models)
466     plot_learning_curves(histories, title="Curvas conjuntas de
467     entrenamiento/validación (DL)")
468     bagging_metrics = compute_metrics(y_test, bagging_preds)
469     print("Métricas Bagging (H=4):", {k: round(v,4) for k,v in metrics.
470     items()})
471
472     # PREDICCIÓN FUTURA FINAL
473     print("\n=== Predicción futura (última ventana de cada empresa) ===")
474
475     n_companies = df_sub.shape[1]
476     y_pred_last = np.zeros((n_companies, H)) # Asegurar matriz 2D
477
478     # Entrenamos cada modelo una vez con todos los datos
479     trained_models = {}
480     for name in top_models:
481         if name in ["XGB", "LGBM"]:
482             model = MultiOutputRegressor(clone(best_models[name])) if H >
483             1 else clone(best_models[name])
484             model.fit(X_full_scaled, y_full)
485         else:
486             config = best_models[name]
487             X_full_seq = X_full_scaled.reshape((X_full_scaled.shape[0], W
488             , 1))
489             model = dl_models[name](input_shape=(W,1), H=H, **config)
490             es = EarlyStopping(monitor='val_loss', patience=3,
491             restore_best_weights=True)
492             lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5,
493             patience=2)
494             model.fit(X_full_seq, y_full, epochs=20, batch_size=32,
495             verbose=0, callbacks=[es, lr])
496             trained_models[name] = model
497
498     # Predicción empresa por empresa
499     for i, company in enumerate(df_sub.columns):
500         series = df_sub[company].values
501         last_window = series[-W:].reshape(1, -1)
502         last_window_scaled = scaler_full.transform(last_window)
503
504         preds_models = []
505         for name, model in trained_models.items():
506             if name in ["XGB", "LGBM"]:
507                 pred = model.predict(last_window_scaled)
508             else: # DL

```

```

501         last_seq = last_window_scaled.reshape((1,W,1))
502         pred = model.predict(last_seq, verbose=0)
503         preds_models.append(pred.reshape(-1)) # Aseguramos (H,)
504
505     # Bagging simple: promedio entre modelos
506     y_pred_last[i] = np.mean(preds_models, axis=0)
507
508
509     # VISUALIZACIÓN PARA H=4
510     print("\n== Predicciones futuras (4 semanas) ==")
511
512     # Seleccionamos las 6 empresas con mayor crecimiento previsto en el ú
513     ltimo paso (semana 4)
514     last_values = df_sub.iloc[-1].values
515     diff_abs = y_pred_last[:, -1] - last_values
516     diff_pct = (diff_abs / last_values) * 100
517     top6_idx = np.argsort(diff_pct)[-6:][::-1]
518
519     for i in top6_idx:
520         company = df_sub.columns[i]
521         current = last_values[i]
522         preds = y_pred_last[i]
523         print(f"\nEmpresa: {company}")
524         print(f"    Ultimo valor: {current:.2f}")
525         for h, pred in enumerate(preds, start=1):
526             abs_diff = pred - current
527             pct_diff = (abs_diff / current) * 100
528             print(f"    +{h} semana(s): Pred={pred:.2f} | Dif={abs_diff:.2
529             f} | Dif%={pct_diff:.2f}%")
530
531     # GRAFICO HISTÓRICO + PREDICCIONES
532     fig, axes = plt.subplots(3, 2, figsize=(14, 12))
533     axes = axes.flatten()
534
535     for j, idx in enumerate(top6_idx):
536         company = df_sub.columns[idx]
537         series = df_sub[company]
538
539         # Histórico
540         axes[j].plot(series.index, series.values, label="Histórico",
541         color="black")
542
543         # Fechas futuras
544         future_dates = [series.index[-1] + pd.Timedelta(weeks=k) for k in
545         range(1, H+1)]
546
547         # Unimos último valor real + predicciones
548         full_dates = [series.index[-1]] + future_dates
549         full_preds = [series.values[-1]] + list(y_pred_last[idx])
550
551         # Graficamos predicciones unidas

```

```
548         axes[j].plot(full_dates, full_preds, "ro-", label="Predicción (4  
549             semanas)")  
550         axes[j].set_title(company)  
551         axes[j].legend()  
552         axes[j].grid(True)  
553  
554     plt.tight_layout()  
555     plt.show()
```

D. Código combinado con H=1 y H=4

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.metrics import mean_squared_error, r2_score,
mean_absolute_error, explained_variance_score
7 from sklearn.multioutput import MultiOutputRegressor
8 from sklearn.model_selection import ParameterGrid, TimeSeriesSplit
9 from sklearn.base import clone
10
11 from xgboost import XGBRegressor
12 from lightgbm import LGBMRegressor
13 from tensorflow.keras.models import Sequential
14 from tensorflow.keras.layers import LSTM, GRU, Conv1D, Flatten, Dropout,
Dense
15 from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
16 from itertools import product
17
18 import warnings
19 warnings.filterwarnings("ignore")
20
21 SEED = 42
22 np.random.seed(SEED)
23
24 def create_supervised_with_dates(df, W=25, H=1):
25     """
26     Genera un conjunto de datos supervisado a partir de una serie
27     temporal,
28     utilizando ventanas deslizantes y manteniendo las fechas
29     correspondientes.
30
31     Parameters
32     -----
33     df : pandas.DataFrame
34         DataFrame con índices de fechas y columnas de series temporales.
35     W : int, optional (default=25)
36         Longitud de la ventana de entrada (número de observaciones
37         pasadas).
38     H : int, optional (default=1)
39         Horizonte de predicción (número de pasos a predecir).
40
41     Returns
42     -----
43     X : numpy.ndarray
44         Matriz de entradas con las ventanas de tamaño W.
45     y : numpy.ndarray
46         Matriz de salidas con los valores futuros a predecir.

```

```

44     dates : numpy.ndarray
45         Fechas correspondientes a cada salida en y.
46     """
47     X, y, dates = [], [], []
48     for col in df.columns:
49         data = df[col].values
50         idx = df.index
51         for i in range(len(data) - W - H + 1):
52             X.append(data[i:i+W])
53             y.append(data[i+W:i+W+H])
54             dates.append(idx[i+W:i+W+H])
55     return np.array(X), np.array(y), np.array(dates)
56
57
58 def compute_metrics(y_true, y_pred):
59     """
60     Calcula varias métricas de evaluación para modelos de predicción.
61
62     Parameters
63     -----
64     y_true : array-like
65         Valores reales de la serie.
66     y_pred : array-like
67         Valores predichos por el modelo.
68
69     Returns
70     -----
71     dict
72         Diccionario con las métricas calculadas:
73         - "MSE" : Error cuadrático medio.
74         - "RMSE": Raíz del error cuadrático medio.
75         - "MAE" : Error absoluto medio.
76         - "R2"  : Coeficiente de determinación.
77         - "MAPE": Error porcentual absoluto medio.
78         - "SMAPE": Error porcentual absoluto medio simétrico.
79         - "EVS" : Varianza explicada.
80     """
81     y_true = np.ravel(y_true)
82     y_pred = np.ravel(y_pred)
83     mse = mean_squared_error(y_true, y_pred)
84     rmse = np.sqrt(mse)
85     mae = mean_absolute_error(y_true, y_pred)
86     r2 = r2_score(y_true, y_pred)
87     mape = np.mean(np.abs((y_true - y_pred)/(y_true+1e-6))) * 100
88     smape = 100*np.mean(2*np.abs(y_pred-y_true)/(np.abs(y_true)+np.abs(y_pred)+1e-6))
89     evs = explained_variance_score(y_true, y_pred)
90     return {"MSE":mse,"RMSE":rmse,"MAE":mae,"R2":r2,"MAPE":mape,"SMAPE":smape,"EVS":evs}
91
92 # MODEL BUILDERS

```

```

93 def build_lstm(input_shape, units=16, dropout=0.3, H=1, **kwargs):
94     """
95     Construye un modelo LSTM para predicción de series temporales.
96
97     Parameters
98     -----
99     input_shape : tuple
100         Forma de la entrada (timesteps, features).
101     units : int, optional (default=16)
102         Número de unidades en la capa LSTM.
103     dropout : float, optional (default=0.3)
104         Tasa de dropout para regularización.
105     H : int, optional (default=1)
106         Horizonte de predicción (número de pasos a predecir).
107     **kwargs : dict
108         Argumentos adicionales para compatibilidad.
109
110     Returns
111     -----
112     keras.Sequential
113         Modelo LSTM compilado con optimizador Adam y pérdida MSE.
114     """
115     model = Sequential([LSTM(units, input_shape=input_shape),
116                        Dropout(dropout),
117                        Dense(H)])
118     model.compile(optimizer='adam', loss='mse')
119     return model
120
121 def build_gru(input_shape, units=16, dropout=0.3, H=1, **kwargs):
122     """
123     Construye un modelo GRU para predicción de series temporales.
124
125     Parameters
126     -----
127     input_shape : tuple
128         Forma de la entrada (timesteps, features).
129     units : int, optional (default=16)
130         Número de unidades en la capa GRU.
131     dropout : float, optional (default=0.3)
132         Tasa de dropout para regularización.
133     H : int, optional (default=1)
134         Horizonte de predicción (número de pasos a predecir).
135     **kwargs : dict
136         Argumentos adicionales para compatibilidad.
137
138     Returns
139     -----
140     keras.Sequential
141         Modelo GRU compilado con optimizador Adam y pérdida MSE.
142     """
143     model = Sequential([GRU(units, input_shape=input_shape),

```

```

144         Dropout(dropout),
145         Dense(H)])
146     model.compile(optimizer='adam', loss='mse')
147     return model
148
149 def build_conv1d(input_shape, units=16, dropout=0.3, kernel_size=3, H=1,
150 **kwargs):
151     """
152     Construye un modelo Conv1D para predicción de series temporales.
153
154     Parameters
155     -----
156     input_shape : tuple
157         Forma de la entrada (timesteps, features).
158     units : int, optional (default=16)
159         Número de filtros en la capa Conv1D.
160     dropout : float, optional (default=0.3)
161         Tasa de dropout para regularización.
162     kernel_size : int, optional (default=3)
163         Tamaño de la ventana de convolución.
164     H : int, optional (default=1)
165         Horizonte de predicción (número de pasos a predecir).
166     **kwargs : dict
167         Argumentos adicionales para compatibilidad.
168
169     Returns
170     -----
171     keras.Sequential
172         Modelo Conv1D compilado con optimizador Adam y pérdida MSE.
173     """
174     model = Sequential([Conv1D(filters=units, kernel_size=kernel_size,
175     activation='relu', input_shape=input_shape),
176     Flatten(),
177     Dropout(dropout),
178     Dense(H)])
179     model.compile(optimizer='adam', loss='mse')
180     return model
181
182 dl_models = {"LSTM": build_lstm, "GRU": build_gru, "Conv1D": build_conv1d}
183
184 # CROSS VALIDATION
185 def tscv_ml(model, X, y, n_splits=5):
186     """
187     Realiza validación cruzada en series temporales para modelos de
188     machine learning.
189
190     Parameters
191     -----
192     model : estimator
193         Modelo base a entrenar (compatible con scikit-learn).

```

```

191     X : numpy.ndarray
192         Conjunto de entradas, de forma (n_muestras, ventana, features).
193     y : numpy.ndarray
194         Conjunto de salidas, de forma (n_muestras, horizonte).
195     n_splits : int, optional (default=5)
196         Número de divisiones para TimeSeriesSplit.
197
198     Returns
199     -----
200     dict
201         Diccionario con el promedio de métricas calculadas en cada split:
202         - "MSE" : Error cuadrático medio.
203         - "RMSE" : Raíz del error cuadrático medio.
204         - "MAE" : Error absoluto medio.
205         - "R2" : Coeficiente de determinación.
206         - "MAPE" : Error porcentual absoluto medio.
207         - "SMAPE" : Error porcentual absoluto medio simétrico.
208         - "EVS" : Varianza explicada.
209     """
210     X_flat = X.reshape((X.shape[0], -1))
211     tscv = TimeSeriesSplit(n_splits=n_splits)
212     metrics_list = []
213     for train_idx, test_idx in tscv.split(X_flat):
214         X_train, X_test = X_flat[train_idx], X_flat[test_idx]
215         y_train, y_test = y[train_idx], y[test_idx]
216         scaler = StandardScaler()
217         X_train_scaled = scaler.fit_transform(X_train)
218         X_test_scaled = scaler.transform(X_test)
219         model_fold = MultiOutputRegressor(clone(model)) if y.shape[1] > 1
220         else clone(model)
221         model_fold.fit(X_train_scaled, y_train if y_train.shape[1] > 1
222         else np.ravel(y_train))
223         y_pred = model_fold.predict(X_test_scaled)
224         metrics_list.append(compute_metrics(y_test, y_pred))
225     return {k: np.mean([m[k] for m in metrics_list]) for k in
226     metrics_list[0]}
227
228 def tscv_dl(model_builder, X, y, n_splits=3, units=16, dropout=0.3,
229 epochs=10, batch_size=32, kernel_size=3):
230     """
231     Realiza validación cruzada en series temporales para modelos de deep
232     learning.
233
234     Parameters
235     -----
236     model_builder : callable
237         Función constructora del modelo (ej. build_lstm, build_gru,
238         build_conv1d).
239     X : numpy.ndarray
240         Conjunto de entradas de forma (n_muestras, ventana).

```

```

236     y : numpy.ndarray
237         Conjunto de salidas de forma (n_muestras, horizonte).
238     n_splits : int, optional (default=3)
239         Número de divisiones para TimeSeriesSplit.
240     units : int, optional (default=16)
241         Número de unidades/neurona en la capa principal del modelo.
242     dropout : float, optional (default=0.3)
243         Tasa de dropout para regularización.
244     epochs : int, optional (default=10)
245         Número máximo de épocas de entrenamiento.
246     batch_size : int, optional (default=32)
247         Tamaño del lote para el entrenamiento.
248     kernel_size : int, optional (default=3)
249         Tamaño de kernel (solo usado en modelos Conv1D).
250
251     Returns
252     -----
253     dict
254         Diccionario con el promedio de métricas calculadas en cada split:
255         - "MSE" : Error cuadrático medio.
256         - "RMSE" : Raíz del error cuadrático medio.
257         - "MAE" : Error absoluto medio.
258         - "R2" : Coeficiente de determinación.
259         - "MAPE" : Error porcentual absoluto medio.
260         - "SMAPE" : Error porcentual absoluto medio simétrico.
261         - "EVS" : Varianza explicada.
262     """
263     X_seq = X.reshape((X.shape[0], X.shape[1], 1))
264     tscv = TimeSeriesSplit(n_splits=n_splits)
265     metrics_list = []
266     for train_idx, test_idx in tscv.split(X_seq):
267         X_train, X_test = X_seq[train_idx], X_seq[test_idx]
268         y_train, y_test = y[train_idx], y[test_idx]
269         scaler = StandardScaler()
270         X_train_2d = X_train.reshape((X_train.shape[0], -1))
271         X_test_2d = X_test.reshape((X_test.shape[0], -1))
272         X_train_scaled = scaler.fit_transform(X_train_2d).reshape(X_train
273             .shape)
274         X_test_scaled = scaler.transform(X_test_2d).reshape(X_test.shape)
275         model = model_builder(input_shape=(X_train.shape[1], 1),
276             units=units, dropout=dropout, H=y_train.
277             shape[1], kernel_size=kernel_size)
278         es = EarlyStopping(monitor='val_loss', patience=2,
279             restore_best_weights=True)
280         lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience
281             =1)
282         model.fit(X_train_scaled, y_train, validation_data=(X_test_scaled
283             , y_test),
284             epochs=epochs, batch_size=batch_size, verbose=0,
285             callbacks=[es, lr])
286         y_pred = model.predict(X_test_scaled, verbose=0)

```

```

281     metrics_list.append(compute_metrics(y_test, y_pred))
282     return {k: np.mean([m[k] for m in metrics_list]) for k in
283           metrics_list[0]}
284
285 # RUN EXPERIMENT
286
287 def run_experiment(path_csv, W=25, H=1, sub_sample_ratio=0.4):
288     """
289     Ejecuta un experimento de predicción con múltiples modelos (ML y DL),
290     evaluando su rendimiento mediante validación cruzada en series
291     temporales.
292
293     Parameters
294     -----
295     path_csv : str
296         Ruta al archivo CSV con precios históricos (índice temporal y
297         columnas por empresa).
298     W : int, optional (default=25)
299         Tamaño de la ventana temporal utilizada como entrada.
300     H : int, optional (default=1)
301         Horizonte de predicción (número de pasos futuros a estimar).
302     sub_sample_ratio : float, optional (default=0.4)
303         Proporción de columnas/empresas seleccionadas aleatoriamente para
304         el experimento.
305
306     Returns
307     -----
308     results : dict
309         Métricas de rendimiento (R2) obtenidas para cada modelo.
310     best_models : dict
311         Mejor configuración encontrada para cada modelo (hiperparámetros
312         o arquitectura).
313     df_sub : pandas.DataFrame
314         Subconjunto de datos utilizado en el experimento.
315     """
316     np.random.seed(SEED)
317     df_weekly = pd.read_csv(path_csv, index_col=0, parse_dates=True)
318     df_weekly.index = pd.to_datetime(df_weekly.index, utc=True)
319
320     sampled_cols = np.random.choice(df_weekly.columns, int(len(df_weekly.
321           columns)*sub_sample_ratio), replace=False)
322     df_sub = df_weekly[sampled_cols]
323
324     X_sub, y_sub, _ = create_supervised_with_dates(df_sub, W=W, H=H)
325     X_sub, y_sub = np.nan_to_num(X_sub), np.nan_to_num(y_sub)
326
327     xgb_params = {"n_estimators": [100, 200], "max_depth": [3, 5], "
328           learning_rate": [0.01, 0.05]}
329     lgb_params = {"n_estimators": [100, 200], "max_depth": [5, 10], "
330           learning_rate": [0.01, 0.05]}
331     dl_params = {'units': [16, 32], 'dropout': [0.2, 0.3]}

```

```

323     c1d_params = {'units':[16,32], 'dropout':[0.2,0.3], 'kernel_size':
324                   :[3,5]}
325
326     ml_models = {"XGB": XGBRegressor, "LGBM": LGBMRegressor}
327     results, best_models = {}, {}
328
329     # ML
330     for name, ModelClass in ml_models.items():
331         best_score, best_model = -np.inf, None
332         param_grid = xgb_params if name=="XGB" else lgb_params
333         for params in ParameterGrid(param_grid):
334             print(f"\nEntrenando {name} con {params}")
335             model = ModelClass(random_state=SEED, verbosity=0, **params)
336             if name=="XGB" else ModelClass(random_state=SEED, verbose=-1,
337                                             **params)
338             metrics = tscv_ml(model, X_sub, y_sub, n_splits=3)
339             print(f"R2={metrics['R2']:.4f}")
340             if metrics['R2'] > best_score:
341                 best_score, best_model = metrics['R2'], model
342             results[name] = best_score
343             best_models[name] = best_model
344
345     # DL
346     for name, builder in dl_models.items():
347         best_score, best_config = -np.inf, None
348         combos = product(c1d_params['units'], c1d_params['dropout'],
349                          c1d_params['kernel_size']) if name=="Conv1D" else product(
350                          dl_params['units'], dl_params['dropout'])
351         for combo in combos:
352             config = {'units':combo[0], 'dropout':combo[1]} if name!="
353                     Conv1D" else {'units':combo[0], 'dropout':combo[1], '
354                     kernel_size':combo[2]}
355             print(f"\nEntrenando {name} con {config}")
356             metrics = tscv_dl(builder, X_sub, y_sub, n_splits=3, **config
357                               )
358             print(f"R2={metrics['R2']:.4f}")
359             if metrics['R2'] > best_score:
360                 best_score, best_config = metrics['R2'], config
361             results[name] = best_score
362             best_models[name] = best_config
363
364     return results, best_models, df_sub
365
366 # BAGGING AND PREDICTIONS
367 def run_bagging_and_forecast(df_sub, W=25, H=1, best_models=None,
368                             top_models=None, n_bags=5):
369     """
370     Ejecuta un esquema de Bagging con los mejores modelos y genera
371     predicciones futuras.
372
373     Parameters

```

```

364 -----
365 df_sub : pandas.DataFrame
366     Subconjunto de datos con series temporales (una columna por
367     empresa).
368 W : int, optional (default=25)
369     Tamaño de la ventana temporal utilizada como entrada.
370 H : int, optional (default=1)
371     Horizonte de predicción (número de pasos futuros a estimar).
372 best_models : dict, optional
373     Modelos o configuraciones óptimas obtenidas en la fase de bú
374     squeda de hiperparámetros.
375 top_models : list of str, optional
376     Lista de modelos a incluir en el ensamble. Por defecto ["XGB", "
377     Conv1D"].
378 n_bags : int, optional (default=5)
379     Número de repeticiones en el ensamble Bagging por cada modelo.
380
381 Returns
382 -----
383 y_pred_last : numpy.ndarray
384     Predicciones futuras para cada empresa en df_sub.
385     - Forma (n_empresas, H) si H > 1.
386     - Forma (n_empresas,) si H = 1.
387 metrics : dict
388     Métricas de evaluación promedio en el conjunto de test:
389     - "MSE" : Error cuadrático medio.
390     - "RMSE" : Raíz del error cuadrático medio.
391     - "MAE" : Error absoluto medio.
392     - "R2" : Coeficiente de determinación.
393     - "MAPE" : Error porcentual absoluto medio.
394     - "SMAPE" : Error porcentual absoluto medio simétrico.
395     - "EVS" : Varianza explicada.
396 """
397 if top_models is None:
398     top_models = ["XGB", "Conv1D"]
399
400 X, y, _ = create_supervised_with_dates(df_sub, W=W, H=H)
401 X, y = np.nan_to_num(X), np.nan_to_num(y)
402 X_flat = X.reshape((X.shape[0], -1))
403
404 tscv = TimeSeriesSplit(n_splits=5)
405 train_idx, test_idx = list(tscv.split(X_flat))[-1]
406 X_train, X_test = X_flat[train_idx], X_flat[test_idx]
407 y_train, y_test = y[train_idx], y[test_idx]
408
409 scaler = StandardScaler()
410 X_train = scaler.fit_transform(X_train)
411 X_test = scaler.transform(X_test)
412
413 # Bagging
414 bagging_preds = np.zeros_like(y_test, dtype=float)

```

```

412     for name in top_models:
413         preds_bag = np.zeros_like(y_test, dtype=float)
414         for _ in range(n_bags):
415             if name in ["XGB", "LGBM"]:
416                 base = clone(best_models[name])
417                 if H > 1:
418                     model = MultiOutputRegressor(base)
419                     model.fit(X_train, y_train)
420                     preds = model.predict(X_test)
421                 else:
422                     base.fit(X_train, np.ravel(y_train))
423                     preds = base.predict(X_test).reshape(-1,1)
424             else:
425                 X_train_seq = X_train.reshape((X_train.shape[0], W, 1))
426                 X_test_seq = X_test.reshape((X_test.shape[0], W, 1))
427                 config = best_models[name]
428                 model = dl_models[name](input_shape=(W,1), H=H, **config)
429                 es = EarlyStopping(monitor='val_loss', patience=3,
430                                   restore_best_weights=True)
431                 lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5,
432                                       patience=2)
433                 model.fit(X_train_seq, y_train, validation_data=(
434                     X_test_seq, y_test),
435                           epochs=20, batch_size=32, verbose=0, callbacks
436                           =[es, lr])
437                 preds = model.predict(X_test_seq, verbose=0)
438             preds_bag += preds
439         preds_bag /= n_bags
440         bagging_preds += preds_bag
441     bagging_preds /= len(top_models)
442
443     metrics = compute_metrics(y_test, bagging_preds)
444     print(f"Métricas Bagging (H={H}):", {k: round(v,4) for k,v in metrics
445                                           .items()})
446
447     # Reentrenamiento con todo
448     X_full, y_full, _ = create_supervised_with_dates(df_sub, W=W, H=H)
449     X_full, y_full = np.nan_to_num(X_full), np.nan_to_num(y_full)
450     X_full_flat = X_full.reshape((X_full.shape[0], -1))
451     X_full_scaled = scaler.fit_transform(X_full_flat)
452
453     trained_models = {}
454     for name in top_models:
455         if name in ["XGB", "LGBM"]:
456             base = clone(best_models[name])
457             if H > 1:
458                 model = MultiOutputRegressor(base)
459                 model.fit(X_full_scaled, y_full)
460             else:
461                 base.fit(X_full_scaled, np.ravel(y_full))
462             model = base

```

```

458         else:
459             X_full_seq = X_full_scaled.reshape((X_full_scaled.shape[0], W
460             , 1))
461             config = best_models[name]
462             model = dl_models[name](input_shape=(W,1), H=H, **config)
463             es = EarlyStopping(monitor='val_loss', patience=3,
464             restore_best_weights=True)
465             lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5,
466             patience=2)
467             model.fit(X_full_seq, y_full, epochs=20, batch_size=32,
468             verbose=0, callbacks=[es, lr])
469             trained_models[name] = model
470
471         # Predicción futura
472         n_companies = df_sub.shape[1]
473         y_pred_last = np.zeros((n_companies, H)) if H > 1 else np.zeros(
474         n_companies)
475
476         for i, company in enumerate(df_sub.columns):
477             series = df_sub[company].values
478             last_window = series[-W:].reshape(1, -1)
479             last_window_scaled = scaler.transform(last_window)
480
481             preds_models = []
482             for name, model in trained_models.items():
483                 if name in ["XGB", "LGBM"]:
484                     preds_models.append(model.predict(last_window_scaled))
485                 else:
486                     last_seq = last_window_scaled.reshape((1, W, 1))
487                     preds_models.append(model.predict(last_seq, verbose=0))
488             preds_models = [np.ravel(p) for p in preds_models]
489             preds_models = np.vstack(preds_models)
490             y_pred_last[i] = preds_models.mean(axis=0)
491
492         return y_pred_last, metrics
493
494 # RECURSIVE FORECASTING
495 def recursive_forecast_H1(df_sub, trained_model_H1, scaler, W=25, steps
496 =4):
497     """
498     Genera predicciones recursivas a partir de un modelo entrenado para H
499     =1.
500
501     Parameters
502     -----
503     df_sub : pandas.DataFrame
504         Subconjunto de datos con series temporales (una columna por
505         empresa).
506     trained_model_H1 : object
507         Modelo entrenado para horizonte H=1 (puede ser de ML o DL).
508     scaler : sklearn.preprocessing.StandardScaler

```

```

501     Objeto de escalado utilizado en el entrenamiento.
502     W : int, optional (default=25)
503         Tamaño de la ventana temporal utilizada como entrada.
504     steps : int, optional (default=4)
505         Número de pasos futuros a predecir de forma recursiva.
506
507     Returns
508     -----
509     y_pred_recursive : numpy.ndarray
510         Predicciones recursivas para cada empresa, de forma (n_empresas,
511         steps).
512     """
513     n_companies = df_sub.shape[1]
514     y_pred_recursive = np.zeros((n_companies, steps))
515
516     for i, company in enumerate(df_sub.columns):
517         series = df_sub[company].values
518         window = series[-W:].reshape(1, -1)
519         for s in range(steps):
520             window_scaled = scaler.transform(window)
521             if hasattr(trained_model_H1, "predict"): # ML
522                 pred = trained_model_H1.predict(window_scaled)
523             else: # DL
524                 pred = trained_model_H1.predict(window_scaled.reshape((1,
525                 W, 1)), verbose=0)
526             pred = float(np.ravel(pred))
527             y_pred_recursive[i, s] = pred
528             window = np.roll(window, -1)
529             window[0, -1] = pred
530         return y_pred_recursive
531
532 # PLOT FORECASTS
533 def plot_forecasts_full(df_sub, y_pred_last_H1, y_pred_last_H4, H1=1, H4
534 =4, title="Predicciones (Full)":
535     """
536     Genera gráficos de predicciones históricas y futuras (H=1 y H=4)
537     para las 6 empresas con mayor crecimiento absoluto previsto.
538
539     Parameters
540     -----
541     df_sub : pandas.DataFrame
542         Subconjunto de datos con series históricas (una columna por
543         empresa).
544     y_pred_last_H1 : numpy.ndarray
545         Predicciones de horizonte H=1 (un paso adelante).
546     y_pred_last_H4 : numpy.ndarray
547         Predicciones de horizonte H=4 (múltiples pasos).
548     H1 : int, optional (default=1)
549         Horizonte usado en las predicciones H=1.
550     H4 : int, optional (default=4)

```

```

548     Horizonte usado en las predicciones H=4.
549     title : str, optional
550         Título de la figura.
551
552     Returns
553     -----
554     None
555
556     Muestra los gráficos en pantalla.
557     """
558     last_values = df_sub.iloc[-1].values
559     diff_abs = y_pred_last_H4[:, -1] - last_values
560     top6_idx = np.argsort(diff_abs)[-6:][::-1]
561
562     fig, axes = plt.subplots(3, 2, figsize=(14, 10))
563     axes = axes.flatten()
564     for j, idx in enumerate(top6_idx):
565         company = df_sub.columns[idx]
566         series = df_sub[company]
567         last_date = series.index[-1]
568         axes[j].plot(series.index, series.values, color="black", label="
569         Histórico")
570         next_date = last_date + pd.Timedelta(weeks=1)
571         pred_val_H1 = float(y_pred_last_H1[idx])
572         axes[j].plot([last_date, next_date],
573                     [series.values[-1], pred_val_H1],
574                     "bo--", label=f"Pred H={H1}")
575         future_dates = [last_date + pd.Timedelta(weeks=k) for k in range
576                        (1, H4+1)]
577         full_dates = [last_date] + future_dates
578         full_preds = [series.values[-1]] + list(y_pred_last_H4[idx])
579         axes[j].plot(full_dates, full_preds, "ro-", label=f"Pred H={H4}")
580         axes[j].set_title(company)
581         axes[j].legend()
582         axes[j].grid(True)
583     plt.suptitle(title)
584     plt.tight_layout()
585     plt.show()
586
587 def plot_forecasts_zoom(df_sub, y_pred_last_H1, y_pred_last_H4, H1=1, H4
588 =4, title="Predicciones (Zoom)":
589     """
590     Genera gráficos comparativos entre predicciones recursivas H=1 y
591     directas H=4
592     para las 6 empresas con mayor crecimiento absoluto previsto.
593
594     Parameters
595     -----
596     df_sub : pandas.DataFrame
597         Subconjunto de datos con series históricas (una columna por
598         empresa).
599     y_pred_last_H1 : numpy.ndarray

```

```

594     Predicciones recursivas basadas en horizonte H=1.
595     y_pred_last_H4 : numpy.ndarray
596     Predicciones directas basadas en horizonte H=4.
597     H1 : int, optional (default=1)
598         Horizonte usado en las predicciones H=1 recursivas.
599     H4 : int, optional (default=4)
600         Horizonte usado en las predicciones H=4 directas.
601     title : str, optional
602         Título de la figura.
603
604     Returns
605     -----
606     None
607
608     Muestra los gráficos en pantalla.
609     """
610     last_values = df_sub.iloc[-1].values
611     diff_abs = y_pred_last_H4[:, -1] - last_values
612     top6_idx = np.argsort(diff_abs)[-6:][::-1]
613
614     fig, axes = plt.subplots(3, 2, figsize=(14, 10))
615     axes = axes.flatten()
616     for j, idx in enumerate(top6_idx):
617         company = df_sub.columns[idx]
618         series = df_sub[company]
619         last_date = series.index[-1]
620
621         # H=1 recursivo
622         future_dates = [last_date + pd.Timedelta(weeks=k) for k in range
623                        (1, H4+1)]
624         full_dates = [last_date] + future_dates
625         full_preds_H1 = [series.values[-1]] + list(y_pred_last_H1[idx])
626         axes[j].plot(full_dates, full_preds_H1, "bo--", label=f"Pred H=1
627                      recursivo")
628
629         # H=4 directo
630         full_preds_H4 = [series.values[-1]] + list(y_pred_last_H4[idx])
631         axes[j].plot(full_dates, full_preds_H4, "ro-", label=f"Pred H=4
632                      directo")
633
634         axes[j].set_title(company)
635         axes[j].legend()
636         axes[j].grid(True)
637         axes[j].tick_params(axis="x", rotation=45)
638
639     plt.suptitle(title)
640     plt.tight_layout()
641     plt.show()
642
643 # MAIN
644 if __name__ == "__main__":
645     path_csv = "stock_weekly_clean.csv"

```

```

642
643 # ----- H=1 -----
644 print("\n=== Experimento H=1 ===")
645 res1, models1, df_sub1 = run_experiment(path_csv, H=1)
646 y_pred_last_H1, metrics_H1 = run_bagging_and_forecast(df_sub1, W=25,
647 H=1, best_models=models1)
648
649 # Guardamos scaler y modelo H=1 para el forecast recursivo
650 X_full, y_full, _ = create_supervised_with_dates(df_sub1, W=25, H=1)
651 X_full = np.nan_to_num(X_full)
652 X_full_flat = X_full.reshape((X_full.shape[0], -1))
653 scaler_H1 = StandardScaler().fit(X_full_flat)
654
655 best_model_name = max(res1, key=res1.get)
656 if best_model_name in ["XGB", "LGBM"]:
657     trained_model_H1 = clone(models1[best_model_name])
658     trained_model_H1.fit(scaler_H1.transform(X_full_flat), np.ravel(
659 y_full))
660 else:
661     X_full_seq = scaler_H1.transform(X_full_flat).reshape((X_full.
662 shape[0], 25, 1))
663     config = models1[best_model_name]
664     trained_model_H1 = dl_models[best_model_name](input_shape=(25,1),
665 H=1, **config)
666     es = EarlyStopping(monitor='val_loss', patience=3,
667 restore_best_weights=True)
668     lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience
669 =2)
670     trained_model_H1.fit(X_full_seq, y_full, epochs=20, batch_size
671 =32, verbose=0, callbacks=[es, lr])
672
673 y_pred_last_H1_recursive = recursive_forecast_H1(df_sub1,
674 trained_model_H1, scaler_H1, W=25, steps=4)
675
676 # ----- H=4 -----
677 print("\n=== Experimento H=4 ===")
678 res4, models4, df_sub4 = run_experiment(path_csv, H=4)
679 y_pred_last_H4, metrics_H4 = run_bagging_and_forecast(df_sub4, W=25,
680 H=4, best_models=models4)
681
682 # ----- Comparaciones -----
683 print("\nComparacion R2 medio por modelo:")
684 for k in set(res1.keys()) | set(res4.keys()):
685     print(f"{k}: H=1: {res1.get(k, '-')}, H=4: {res4.get(k, '-')}")
686
687 print("\n=== Gráfico comparando H=1 recursivo vs H=4 directo ===")
688 plot_forecasts_zoom(df_sub4, y_pred_last_H1_recursive, y_pred_last_H4
689 ,
690
691 H1=1, H4=4, title="Zoom: H=1 recursivo vs H=4
692 directo")

```